

PZ-ENC28J60 以太网模块开发手册

第 1 章 网络协议简介

在学习 LwIP 之前，我们先来了解下网络协议相关知识，通过本章的学习，让大家对网络协议更加了解，为后面 LwIP 的学习做好铺垫。本章分为如下几部分内容：

- 1.1 常用网络协议
- 1.2 网络协议的分层模型
- 1.3 协议层报文间的封装与拆封

1.1 常用网络协议

互联网对人类社会产生的巨大变革，大家是有目共睹的，它几乎改变了人类生活的方方面面。互联网通信的本质是数字通信，任何数字通信都离不开通信协议的制定，通信设备只有按照约定的、统一的方式去封装和解析信息，才能实现通信。互联网通信所要遵守的众多协议，被统称为 TCP/IP。

TCP/IP 是一个协议族，包含众多的协议。但对于网络应用开发人员，可能听到更多的是其中的应用层协议，比如 HTTP、FTP、MQTT 等。

HTTP 协议是 Hyper Text Transfer Protocol（超文本传输协议）的缩写，HTTP 的应用最为广泛。比如大家日常使用电脑时的一个常规操作：打开电脑，打开浏览器，输入网址，最后按下回车，这一刻你就开启了 HTTP 通信。HTTP 协议工作于<客户端-服务端>架构之上，（服务端也称作为服务器端，除非特别说明，否则本书出现的“服务端”即为“服务器端”），浏览器作为 HTTP 客户端通过 URL 向 HTTP 服务端即 WEB 服务器发送所有请求。Web 服务器根据接收到的请求后，向客户端发送响应信息。借助这种浏览器和服务器之间的 HTTP 通信，我们能够足不出户地获得来自世界各地的信息。另外，网页不仅仅是大型服务器的专利，在物联网风潮盛行的今天，许多随处可见的小型设备（空调、冰箱、插座、路由器等）都内嵌网页，在物理链路畅通的情况下，用户可以用手机、平板电脑上的浏览器随时随地监控这些设备。

FTP（File Transfer Protocol）是文件传输协议的简称。FTP 是工作在应用层的网络协议。FTP 使得主机间可以共享文件，用于在两台设备之间传输文件（双向传输）。它也是一个客户端-服务端框架系统。用户可以通过一个支持 FTP 协议的客户端程序，连接到在远程主机上的 FTP 服务端程序，通过客户端程序向服务端程序发出命令，服务端程序执行用户所发出的命令，并将执行的结果返回到客户机。FTP 除了基本的文件上传/下载功能外，还有目录操作、权限设置、身份验证机制，许多网盘的文件传输功能都是基于 FTP 实现的。

在物联网发展的初期，物联网场景中的设备使用何种应用层协议进行通信一直是备受争议的话题。很多开发人员习惯了网页的开发模式，于是经常选择 HTTP

作为通信方式。使用 HTTP 有以下不利因素：HTTP 是一种同步协议，设备需要等待服务器的响应才可以进行下一步的工作，然而在设备数量多、网络不可靠的场景下，实现同步通信很困难；HTTP 是单向的，设备只能主动向服务器发出数据，无法被动的接收来自网络的数据，这不适用于实时控制的场合；HTTP 是有许多帧头和规则的重量级协议，实现在设备中需要耗费大量的系统资源。基于上述的形势，MQTT 和 COAP 等轻量级、异步的通信协议便得到了物联网设备开发者的宠爱，尤其是 MQTT。MQTT（消息队列遥测传输）是 IBM 公司于 1990 年设计并推出的一款通信协议，于 2014 年正式成为了一个 OASIS 开放标准。近年来，MQTT 的应用呈现出爆炸性的增长势头，大有一统物联网的趋势。另外，MQTT 在物联网以外的其他领域也得到了广泛的应用，比如许多公司在制作手机 APP 时，会使用 MQTT 来实现消息推送、即时聊天等功能。

嵌入式设备接入互联网的需求越来越大，有以下几点原因：

（1）近些年，各种带网络接入功能的 MCU、SoC 层出不穷，开源轻量的 TCP/IP 协议栈日趋成熟和完善，云平台的市场越来越繁荣，这些因素大大降低了嵌入式设备的入网成本，也为许多资源受限的低端设备接入互联网提供了可能。

（2）“物联网+”的风潮日渐盛行，设备能够被远程监控，这一点已经成为许多产品的技术要求。

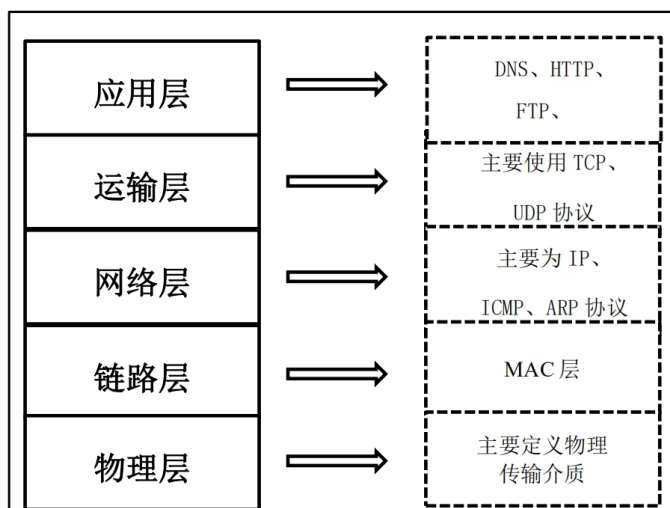
（3）人们对于设备“智能性”的追求越来越高，当今热门的大数据、图像处理、语音识别、机器学习等功能都可以被集成在云端，成为云平台能提供的服务。终端设备大多是计算、存储能力有限的设备，这些设备如果想要获取“智能”，最便捷的办法就是接入云平台，利用各项云服务。

互联网的基础就是 TCP/IP。TCP/IP 是一个非常复杂的协议族，即便我们能把它的设计思想和实现原理都解释得清清楚楚，你也不见得有时时间和精力去学习它，所以本书的写作重点不在于对 TCP/IP 的解读，而在于对它的应用。另外，TCP/IP 的复杂性也决定了它并不是那么简单就能用好的东西，即便我们只关注应用开发，也依然需要对它的许多概念和设计思想有所了解，才能编写出正确、高效、健壮性好的应用程序。

希望能借本攻略，让嵌入式开发工程师们以浓厚的兴趣和清晰的视野，搭上物联网发展的快车。

1.2 网络协议的分层模型

TCP/IP 是一个庞大的协议族，它是众多网络协议的集合，包括：ARP、IP、ICMP、UDP、TCP、DNS、DHCP、HTTP、FTP、MQTT 等等。这些协议按照功能，可以被划分为几个不同的层次，如下图所示：



我们在上一节中介绍的 HTTP、FTP、MQTT，它们隶属于应用层。那么 TCP/IP 为什么需要分层，分层又是依靠什么依据呢？

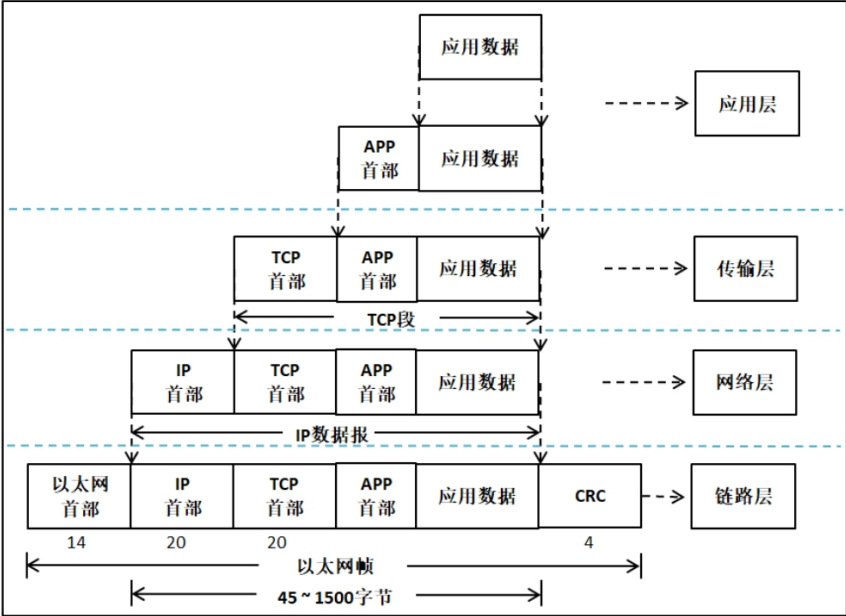
TCP/IP 协议栈中不同协议所完成的功能是不一样的，某些协议的实现要依赖于其它协议，依据这种依赖关系，可以将协议栈分层。在上图中，低层协议为相邻的上层协议提供服务，是上层协议得以实现的基础。

其中，物理层（PHY）规定了传输信号所需要的物理电平、介质特征；链路层（MAC）规定了数据帧能被网卡接收的条件，最常见的方式是利用网卡的 MAC 地址，发送方会在欲发送的数据帧的首部加上接收方网卡的 MAC 地址信息，接收方只有监听到属于自己的 MAC 地址信息后，才会去接收并处理该数据；每台网络设备都应该有自己的网络地址，网络层规定了主机的网络地址该如何定义，以及如何在网络地址和 MAC 地址之间进行映射，即 ARP 协议；网络层实现了数据包在主机之间的传递，而一台主机内部可能运行着多个网络程序，传输层可以区分数据包是属于哪一个应用程序的，可以说传输层实现了数据包端到端的传递。另外，数据包在传输过程中可能会出现丢包、乱序和重复的现象，网络层并没有提供应对这些错误的机制，而传输层可以解决这些问题，如 TCP 协议；应用层以下的工作完成了数据的传递工作，应用层则决定了你如何应用和处理这些数据，

之所以会有许多的应用层协议，是因为互联网中传递的数据种类很多、差异很大、应用场景十分多样。

1.3 协议层报文间的封装与拆封

本书的后面章节会对 TCP/IP 协议栈中的每层协议进行分析和讲解。在这里，我们以下图简单解释一下在数据的发送和接收过程中，TCP/IP 都做了哪些事。



当用户发送数据时，将数据向下交给传输层，这是处于应用层的操作，应用层可以通过调用传输层的接口来编写特定的应用程序。而 TCP/IP 协议一般也会包含一些简单的应用程序如 Telnet 远程登录、FTP 文件传输、SMTP 邮件传输协议等。传输层会在数据前面加上传输层首部（此处以 TCP 协议为例，上图中的传输层首部为 TCP 首部，也可以是 UDP 首部），然后向下交给网络层。同样地，网络层会在数据前面加上网络层首部（IP 首部），然后将数据向下交给链路层，链路层会对数据进行最后一次封装，即在数据前面加上链路层首部（此处使用以太网接口为例），然后将数据交给网卡。最后，网卡将数据转换成物理链路上的电平信号，数据就这样被发送到了网络中。数据的发送过程，可以概括为 TCP/IP 的各层协议对数据进行封装的过程，如上图所示。

当设备的网卡接收到某个数据包后，它会将其放置在网卡的接收缓存中，并告知 TCP/IP 内核。然后 TCP/IP 内核就开始工作了，它会将数据包从接收缓存中取出，并逐层解析数据包中的协议首部信息，并最终将数据交给某个应用程序。

数据的接收过程与发送过程正好相反,可以概括为 TCP/IP 的各层协议对数据进行解析的过程。

普中STM32开发板

第 2 章 LwIP 简介

本章将向大家介绍 LwIP，揭开它的神秘面纱。通过本章的学习，让大家对 LwIP 更加了解，为后面 LwIP 的学习做好铺垫。本章分为如下几部分内容：

- 2.1 LwIP 的优缺点
- 2.2 LwIP 的文件说明
- 2.3 查看 LwIP 的说明文档
- 2.4 使用 vscode 查看源码
- 2.5 LwIP 源码里的 example
- 2.6 LwIP 的三种编程接口

2.1 LwIP 的优缺点

本书以 LwIP 1.4.1 为主要对象进行讲解，后续中出现的 LwIP 如果没有特殊声明，均指 1.4.1 版本。这个版本不是最新的，对于学习而言，大家其实不必纠结于是否必须用最新的版本。LwIP 1.4.1 版本官方下载链接：

<http://savannah.nongnu.org/projects/lwip/>

LwIP 全名：Light weight IP，意思是轻量化的 TCP/IP 协议，是瑞典计算机科学院(SICS)的 Adam Dunkels 开发的一个小型开源的 TCP/IP 协议栈。LwIP 的设计初衷是：用少量的资源消耗实现一个较为完整的 TCP/IP 协议栈，其中“完整”主要指的是 TCP 协议的完整性，实现的重点是在保持 TCP 协议主要功能的基础上减少对 RAM 的占用。此外 LwIP 既可以移植到操作系统上运行，也可以在没有操作系统的情况下独立运行。

(1) LwIP 具有主要特性：

1. 支持 ARP 协议（以太网地址解析协议）。
2. 支持 ICMP 协议（控制报文协议），用于网络的调试与维护。
3. 支持 IGMP 协议（互联网组管理协议），可以实现多播数据的接收。
4. 支持 UDP 协议（用户数据报协议）。
5. 支持 TCP 协议（传输控制协议），包括阻塞控制、RTT 估算、快速恢复和快速转发。
6. 支持 PPP 协议（点对点通信协议），支持 PPPoE。
7. 支持 DNS（域名解析）。
8. 支持 DHCP 协议，动态分配 IP 地址。
9. 支持 IP 协议，包括 IPv4、IPv6 协议，支持 IP 分片与重装功能，多网络接口下的数据包转发。
10. 支持 SNMP 协议（简单网络管理协议）。
11. 支持 AUTOIP，自动 IP 地址配置。
12. 提供专门的内部回调接口(Raw API)，用于提高应用程序性能。
13. 提供可选择的 Socket API、NETCONN API（在多线程情况下使用）。

(2) LwIP 在嵌入式中使用有以下优点：

1. 资源开销低，即轻量化。LwIP 内核有自己的内存管理策略和数据包管理

策略，使得内核处理数据包的效率很高。另外，LwIP 高度可剪裁，一切不需要的功能都可以通过宏编译选项去掉。LwIP 的流畅运行需要 40KB 的代码 ROM 和几十 KB 的 RAM，这让它非常适合用在内存资源受限的嵌入式设备中。

2. 支持的协议较为完整。几乎支持 TCP/IP 中所有常见的协议，这在嵌入式设备中早已够用。

3. 实现了一些常见的应用程序：DHCP 客户端、DNS 客户端、HTTP 服务器、MQTT 客户端、TFTP 服务器、SNTP 客户端等等。

4. 同时提供了三种编程接口：RAW API、NETCONN API（注：NETCONN API 即为 Sequential API，为了统一，下文均采用 NETCONN API）和 Socket API。这三种 API 的执行效率、易用性、可移植性以及时空开销各不相同，用户可以根据实际需要，平衡利弊，选择合适的 API 进行网络应用程序的开发。

5. 高度可移植。其源代码全部用 C 实现，用户可以很方便地实现跨处理器、跨编译器的移植。另外，它对内核中会使用到操作系统功能的地方进行了抽象，使用了一套自定义的 API，用户可以通过自己实现这些 API，从而实现跨操作系统的移植工作。

6. 开源、免费，用户可以不用承担任何商业风险地使用它。

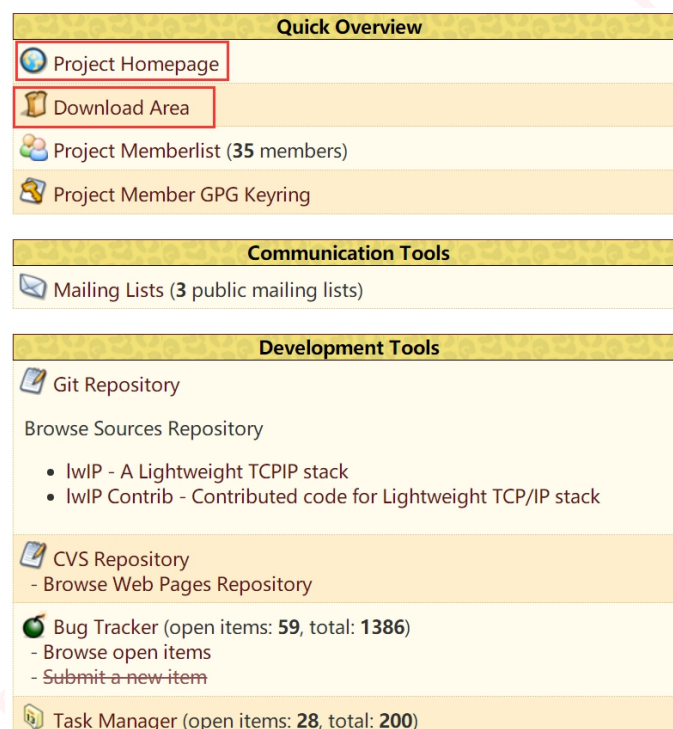
7. 相比于嵌入式领域其它的 TCP/IP 协议栈，比如 uC-TCP/IP、FreeRTOS-TCP 等，LwIP 的发展历史要更悠久一些，得到了更多的验证和测试。LwIP 被广泛用在嵌入式网络设备中，国内一些物联网公司推出的物联网操作系统，其 TCP/IP 核心就是 LwIP；物联网知名的 WiFi 模块 ESP8266，其 TCP/IP 固件，使用的就是 LwIP。

LwIP 尽管有如此多的优点，但它毕竟是为嵌入式而生，所以并没有很完整地实现 TCP/IP 协议栈。相比于 Linux 和 Windows 系统自带的 TCP/IP 协议栈，LwIP 的功能不算完整和强大。但对于大多数物联网领域的网络应用程序，LwIP 已经足够了。

2.2 LwIP 的文件说明

2.2.1 如何获取 LwIP 源码文件

LwIP 的代码已经交给 Savannah 托管，LwIP 的项目主页是：
<http://savannah.nongnu.org/projects/lwip/>。这个主页简单地介绍了一下 LwIP，然后给出了许多链接，你可以通过这些链接去挖掘更多关于 LwIP 的信息。在这里，我们只关注两个地方，如下图中的方框所示。



点击“Project Homepage”，会得到一个网页，如下图所示。

lwIP

2.1.0

Lightweight IP stack

Main Page

Related Pages

Modules

Data Structures

Files

lwIP to disable panel

System initialization

Upgrading

Changelog

How to contribute to lwIP

Common pitfalls

Reporting bugs

Zero-copy RX

System initialization

Multithreading

Optimization hints

Deprecated List

Modules

Data Structures

Files

Overview

INTRODUCTION

lwIP is a small independent implementation of the TCP/IP protocol suite.

The focus of the lwIP TCP/IP implementation is to reduce the RAM usage while still having a full scale TCP. This making lwIP suitable for use in embedded systems with tens of kilobytes of free RAM and room for around 40 kilobytes of code ROM.

lwIP was originally developed by Adam Dunkels at the Computer and Networks Architectures (CNA) lab at the Swedish Institute of Computer Science (SICS) and is now developed and maintained by a worldwide network of developers.

FEATURES

- * IP (Internet Protocol, IPv4 and IPv6) including packet forwarding over multiple network interfaces
- * ICMP (Internet Control Message Protocol) for network maintenance and debugging
- * IGMP (Internet Group Management Protocol) for multicast traffic management
- * MLD (Multicast listener discovery for IPv6). Aims to be compliant with RFC 2710. No support for MLDv2
- * ND (Neighbor discovery and stateless address autoconfiguration for IPv6). Aims to be compliant with RFC 4861 (Neighbor discovery) and RFC 4862

这个网页可以看成是 LwIP 的官方说明文档。我们可以通过这个网页获得关于 LwIP 的很多信息，包括 LwIP 的使用注意、数据的拷贝、系统初始化流程、多线程中要注意的问题、优化方法、内核模块的分类介绍、内核数据结构、内核重要全局变量、内核源码文件等。这些内容专业性比较强，不建议初学时在它上面花费精力，并且里面的很多内容在我们教材的后续章节中会有所讲解。在这里，读者只要知道有这么个东西就行了。

点击“Download Area”，会得到一个网页，如下图所示。

Index of /releases/lwip/

File Name	File Size
Parent directory/	-
drivers/	-
older_versions/	-
contrib-1.4.0.zip	454K
contrib-1.4.0.zip.sig	72
contrib-1.4.1.zip	430K
contrib-1.4.1.zip.sig	287
contrib-2.0.0.RC2.zip	737K
contrib-2.0.0.RC2.zip.sig	287
contrib-2.0.0.zip	533K
contrib-2.0.0.zip.sig	287
contrib-2.0.1.zip	532K
contrib-2.0.1.zip.sig	287
contrib-2.1.0.zip	614K
contrib-2.1.0.zip.sig	287
lwip-1.4.0.zip	600K
lwip-1.4.0.zip.sig	72
lwip-1.4.1.zip	595K
lwip-1.4.1.zip.sig	287
lwip-2.0.0.RC2.zip	3M
lwip-2.0.0.RC2.zip.sig	287
lwip-2.0.0.zip	3M
lwip-2.0.0.zip.sig	287
lwip-2.0.1.zip	3M
lwip-2.0.1.zip.sig	287
lwip-2.0.2.zip	3M
lwip-2.0.2.zip.sig	287
lwip-2.0.3.zip	3M
lwip-2.0.3.zip.sig	287
lwip-2.1.0.zip	4M
lwip-2.1.0.zip.sig	287
lwip-2.1.1.zip	4M
lwip-2.1.1.zip.sig	287
lwip-2.1.2.zip	4M
lwip-2.1.2.zip.sig	287

通过这个网页，我们可以下载到 LwIP 所有版本的源代码包和 contrib 包。你每点击一个红色字体的资源链接，浏览器就会开启一个 ftp 连接，帮助你下载想要的文件到电脑中。但是这个页面提供的下载链接，在国内一般是没有响应的。这个网页最下方的黑字内容推荐我们使用另外一个下载页面：

<http://download-mirror.savannah.gnu.org/releases/>

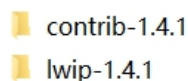
在这个页面下，用户可以下载到所有在 Savannah 托管的开源软件，但我们只关心 LwIP。利用浏览器的搜索功能，快捷键 Ctrl+F，可以快速找到 lwip 目录。在这里为了方便读者，我们直接给出最终的下载链接：

<http://download-mirror.savannah.gnu.org/releases/lwip/>

可能有读者会问，什么是 contrib 包，它与源代码包有什么不同？源代码包里面装的主要是 LwIP 内核的源码文件，而 contrib 包里面装的是移植和应用 LwIP 的一些 demo，即应用示例。contrib 包不属于 LwIP 内核的一部分，里面的很多内容来自开源社区的贡献，因此 contrib 包的版本管理不像内核源码那样严格和规范，但也是很有参考价值的。按理说，LwIP 源码面世越久，开源社区对它的贡献就越大，所以越高版本的 contrib 包，提供的应用示例就越丰富，越有参考价值。在大版本区别不大的情况下，建议大家下载最新的 contrib 包。后续我们会对 contrib 包里面提供的应用示例进行讲解。另外，还有些“.sig”后缀的文件，这是数字签名，大家忽略就好。

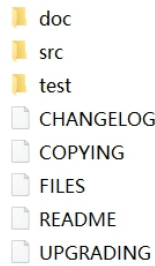
2.2.2 LwIP 文件说明

按照上一小节的介绍，我们下载两个包：lwip-1.4.1.zip（源码包）和 contrib-1.4.1.zip（contrib 包）。解压以后会得到两个文件夹，如下图所示。（在我们资料内已经提供了这两个文件，在“..[LwIP 学习资料](#)”，省去下载的时间）



contrib-1.4.1
lwip-1.4.1

我们先打开“lwip-1.4.1”文件夹，如下图所示。



该目录的内容为：

(1) CHANGELOG 文件记录了 LwIP 在版本升级过程中源代码发生的变化。

(2) COPYING 文件记录了 LwIP 这个开源软件的 license。一个软件开源，不代表你能无限制地使用它，你需要在使用它的过程中遵守一定的规则，这些规则就是 license。大家可以用记事本打开这个 COPYING 文件看看它的内容。开源软件的 license 有很多种，LwIP 的属于 BSD License。LwIP 的开源程度是很高的，你几乎可以无限制地使用它。

(3) FILES 文件用于介绍当前目录下的目录信息。

(4) README 文件对 LwIP 进行了一个简单的介绍。

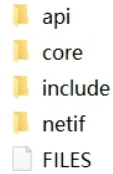
(5) UPGRADING 文件记录了 LwIP 每个大版本的更新，会对用户使用和移植 LwIP 造成的影响。所谓大版本更新指的是：1.3.x-1.4.x-2.0.x-2.1.x。小版本更新，比如 2.0.1-2.0.2-2.0.3，这个过程只是一些 bug 的修复和性能的改善，不会对用户的使用造成影响。用户只要将原有工程的目录中与 LwIP 相关的旧版本文件替换成新版本的文件，重新编译，就能直接使用。

(6) doc 文件夹里面是关于 LwIP 的一些文档，可以看成是应用和移植 LwIP 的指南。但是这些文档比较零散，不成体系，而且纯文本阅读起来很费劲，阅读意义不是很大。

(7) test 文件夹里面是测试 LwIP 内核性能的源码，将它们和 LwIP 源码加入到工程中一起编译，调用它们提供的函数，可以获得许多与 LwIP 内核性能有关的指标。这种内核性能测试功能，只有非常专业的人士才用的到。

(8) src 文件夹里面就是我们最关心的 LwIP 源码文件，下面会详细讲解。

打开 src 文件夹，如下图所示。



api 文件夹里面装的是 NETCONN API 和 Socket API 相关的源文件，只有在操作系统的环境中，才能被编译。

core 文件夹里面是 LwIP 的内核源文件，后续会详细讲解。

include 文件夹里面是 LwIP 所有模块对应的头文件。

netif 文件夹里面是与网卡移植有关的文件，这些文件为我们移植网卡提供了模板，我们可以直接使用。

LwIP 内核是由一系列模块组合而成的，这些模块包括：TCP/IP 协议栈的各种协议、内存管理模块、数据包管理模块、网卡管理模块、网卡接口模块、基础功能类模块、API 模块。每个模块是由相关的几个源文件和头文件组成的，通过头文件对外声明一些函数、宏、数据类型，使得其它模块可以方便地调用此模块的功能。而构成每个模块的头文件都被组织在了 include 目录中，而源文件则根据类型被分散地组织在 api、core、netif 目录中。

接下来，我们介绍一下 core 文件夹，如下图所示。



我们逐一介绍下这些源文件的功能。

ipv4 文件夹里面是与 IPv4 模块相关的源文件，它们实现了 IPv4 协议规定的对数据包的各种操作。ipv4 文件夹中还包括一些并非属于 IP 协议，但会受 IP 协议影响的协议源文件，包括 DHCP、ARP、ICMP、IGMP。

ipv6 文件夹里面是与 IPv6 模块相关的源文件，它们实现了 IPv6 协议规定的对数据包的各种操作。ipv6 文件夹中还包括一些并非属于 IP 协议，但会受 IP 协议影响的协议源文件，包括 DHCP、ARP、ICMP、IGMP。

def.c 文件定义了一些基础类函数，比如主机序和网络序的转换、字符串的查找和比较、整数转换成字符串等，这些函数会被 LwIP 内核的很多模块所调用。在 include 目录里面的 def.h 文件对外声明了 def.c 所实现的函数，同时定义了许多宏，能实现一些基础操作，比如取最大值、取最小值、计算数组长度等，这些宏同样也被内核的许多模块所调用。我们经常可以看到某个内核的源文件在开始的地方#include "def.h"。

dns.c 文件实现了域名解析的功能，有了它，用户就可以在知道服务器域名的情况下，获得该服务器的 IP 地址。很多时候我们只记得服务器域名而不记得服务器 IP 地址，例如“www.baidu.com”就是一个域名，通过 dns 功能，我们就可以得到与服务器域名对应的 IP 地址，这给用户使用带来很大的方便。

init.c 文件对 LwIP 的用户宏配置进行了检查，会将配置错误和不合理的地方，通过编译器的#error 和#warning 功能表示出来。另外，init.c 定义了 lwip_init 初始化函数，这个函数会依次对 LwIP 的各个模块进行初始化。

mem.c 文件实现了动态内存池管理机制，使得 LwIP 内核的各个模块可以灵活地申请和释放内存。

memp.c 文件实现了静态内存堆管理机制，使得 LwIP 内核的各个模块可以快速地申请和释放内存。

netif.c 文件实现了网卡的操作，比如注册/删除网卡、使能/禁能网卡、设置网卡 IP 地址等等。netif.c 与 include 目录中的 netif.h 文件共同构成了 LwIP 的 netif 模块，它对网卡进行了抽象，使得 LwIP 内核可以方便地管理多个特性各异的物理网卡。

pbuf.c 文件实现了 LwIP 对网络数据包的各种操作。网络数据包在 LwIP 内核中以 pbuf 结构体的形式存在，这提高了 LwIP 内核对数据包处理效率，以及提高了数据包在各层之间递交的效率。pbuf 结构体也是我们使用 RAW/Callback API 进行网络应用程序开发的关键，后续我们会详细讲解。

raw.c 文件实现了一个传输层协议的框架，我们可以在它的基础上修改和添

加代码，实现自定义的传输层协议，与 UDP/TCP 一样，它可以与 IP 层直接进行交互。这类似 RAW Socket。在实际的应用中，我们常用 UDP 和 TCP 作为传输层协议。但有时，底层网络开发人员会嫌 UDP 的可靠性太差，或者 TCP 虽然可靠性强，但是很耗费时间和内存，他们需要根据实际需求，平衡利弊，定义自己的传输层协议。LwIP 的 raw 模块可以满足他们的需求。

stats.c 文件实现了 LwIP 内核的统计功能，使用户可以实时地查看 LwIP 内核对网络数据包的处理情况。

sys.c 文件和 **sys.h** 文件构成了 LwIP 的 sys 模块，它提供了与临界区相关的操作。

tcp.c、**tcp_in.c** 和 **tcp_out.c** 文件实现了 TCP 协议，包括对 TCP 连接的操作、对 TCP 数据包的输入输出操作和 TCP 定时器，它们和 include 目录中名称带 tcp 的头文件共同构成了 LwIP 的 TCP 模块。TCP 模块的实现是 LwIP 的最大特点，它以很小的资源开销几乎实现了 TCP 协议中规定的全部内容。TCP 协议是非常复杂的协议，这几个与 TCP 模块相关的文件占据了 LwIP 内核的绝大部分。

timers.c 文件定义了 LwIP 内核的超时处理机制。LwIP 内核中多个模块的实现需要借助超时处理机制，包括 ARP 表项的时间统计、IP 分片报文的重装、TCP 的各种定时器、实现各种应用层协议需要的超时处理。

udp.c 文件实现了 UDP 协议，包括对 UDP 连接的操作和 UDP 数据包的操作。

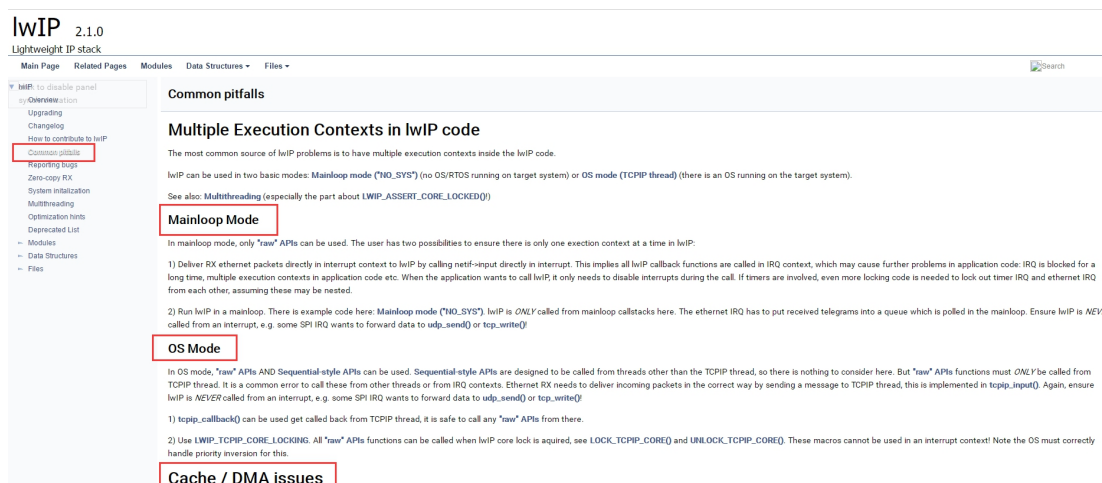
2.3 查看 LwIP 的说明文档

关于 LwIP 的官方说明文档：

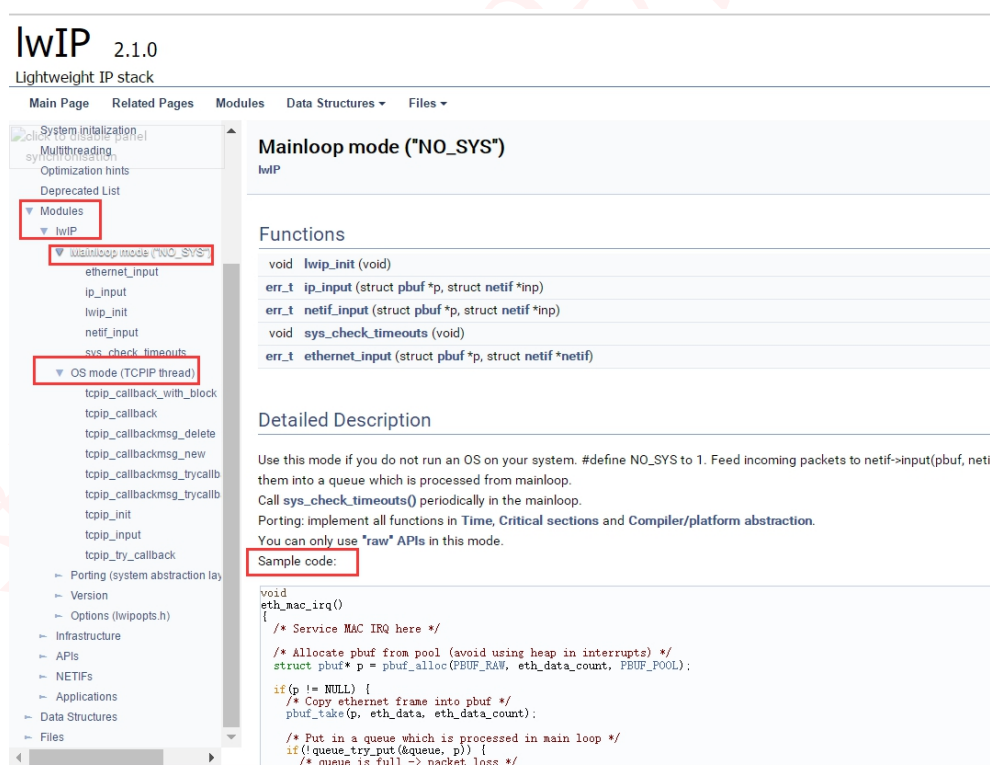
http://www.nongnu.org/lwip/2_1_x/index.html

我就简单带大家浏览一下。打开连接，我们可以看到 LwIP 的 Overview（概述），这里就简单看看即可，我们可以点击左侧的“Common pitfalls”，查看一下 LwIP 常见的陷阱，可能在使用中会遇到，到时候注意一下即可，在前面的章节中，我们也提到过，LwIP 可以工作在无操作系统环境也可以工作在有操作系统的环境中，Common pitfalls 中提到 Mainloop Mode（主函数轮询模式）与

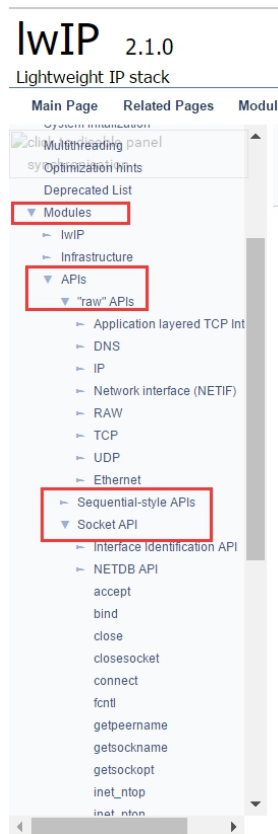
OS Mode（操作系统模式）需要注意的一些事情，具体见下图。



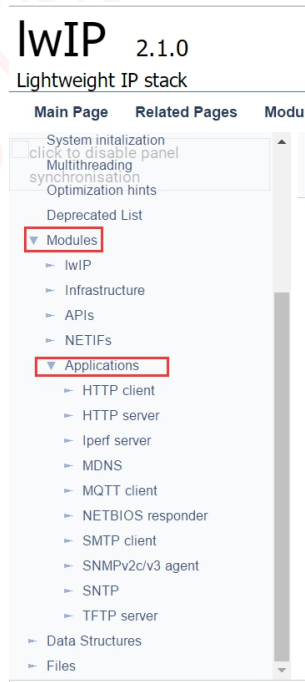
此外，我们还可以点击左侧的“Modules”，查看一些模块相关的说明，以及例子，比如有无操作系统相关的，如，还有基础配置，如 LwIP 的内存管理模块，数据包缓冲区等会是在“Modules ->Infrastructure”页面中，具体见下图。



当然，还有很重要的一些用户常用的 API 函数，也是在“Modules”中可以找到，例如 Raw API，NETCONN API 和 Socket API 等，具体见下图。

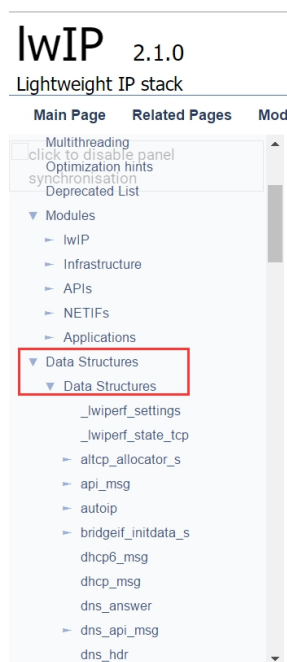


此外还有一些“Applications”应用层相关的说明，如 HTTP、MQTT、TFTP 等，具体见下图。

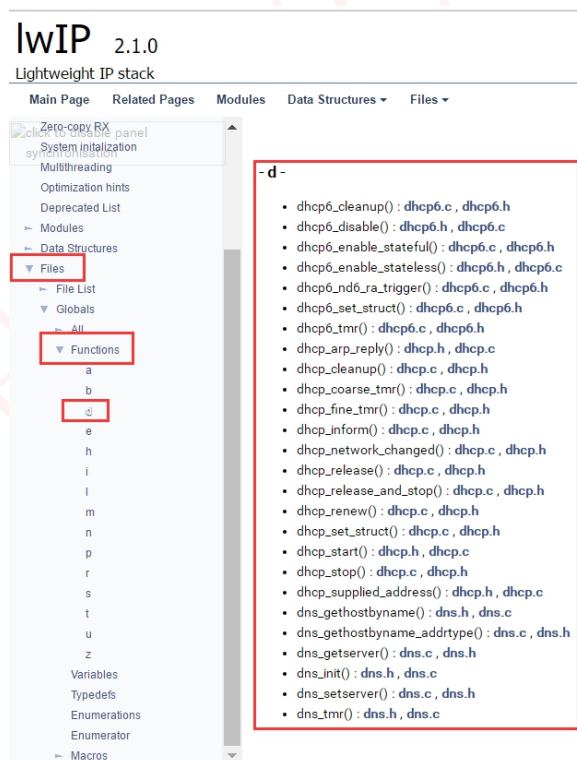


还有一些数据结构相关的说明，当我们在程序中看到哪个数据结构不懂的，都可以在这里找到对应的说明，也是比较重要的，LwIP 本质就是对数据的处理，

其中也使用了大量的数据结构，有空可以多研究研究它，具体见下图。



当然，我们也能通过函数名字的首字母来查找函数的作用，具体见下图。



2.4 使用 vscode 查看源码

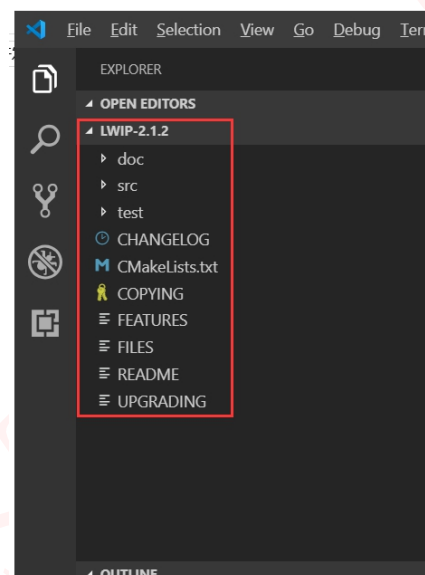
2.4.1 查看文件中的符号列表（函数列表）

LwIP 的源码很庞大，我们使用微软的开源软件——vs code 查看源码，并且快速找到源码的函数与定义，首先我们先安装 vs code，我们可以在

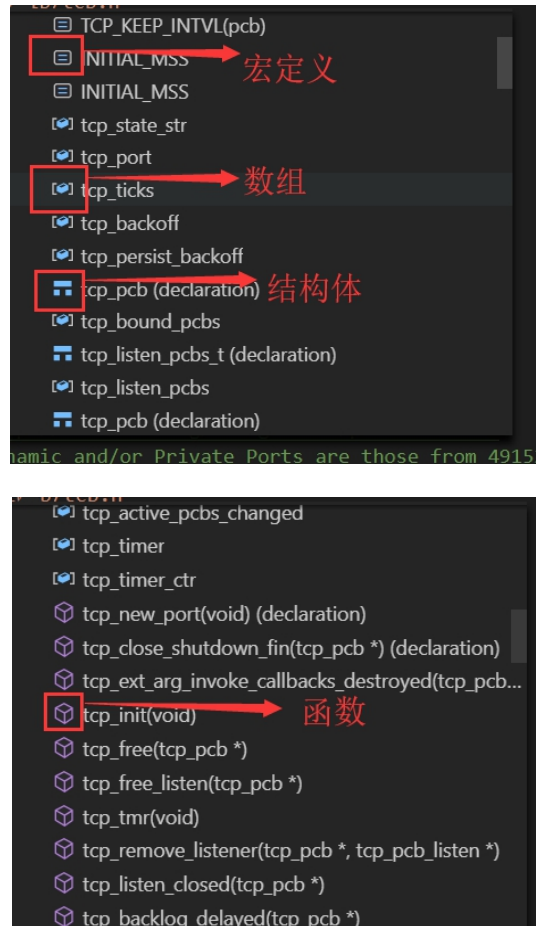
<https://code.visualstudio.com/download>

中下载时候自己电脑的 vs code 版本，然后安装即可。

然后打开安装好的 VS Code，将 LwIP 的源码文件夹拖拽到 VS Code 中，这样子就能直接在 vs code 打开我们整个文件夹的源码了，具体见下图。

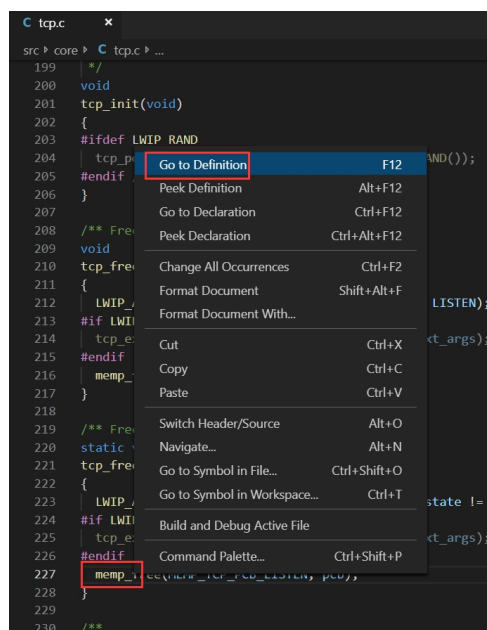


在 vs code 中，就显示了我们打开的源码，LwIP 那么多文件，我们怎么去快速找到源码文件中的某个函数呢？很简单，比如我们知道某个函数的名字的话，可以直接搜索的，这点就不必我多说，但是有时候，我们不记得某个函数的名字，只知道它在哪个文件，或者只知道在好几个文件中的某一个，那么我们就需要一个个去查找这个函数了，vs code 提供很强大的功能，就是可以快速查文件中的符号列表和函数列表，我们首先打开一个源码文件，比如 tcp.c，然后我们通过快捷键“Shift+Ctrl+O”即可打开对应源码文件的符号列表和函数列表，通过查看这些列表，就能知道该源码文件中是否有我们需要的函数或者宏定义等，具体见下图。



2.4.2 函数定义跳转

vs code 看源码是非常方便的，比如，我们可以通过 F12 按键进行跳转到定义，通过“Alt+F12”速览定义，或者通过快捷键“Ctrl+F12”进行 Go to Declaration，这些操作还是很方便的，当然啦，我们也能通过鼠标右键，进行选择，具体见下图。



如果在查看函数之后，想返回跳转前的位置，只需要通过快捷键“Alt+键盘的←（前后左右的左按键）”跳回即可。

2.5 LwIP 源码里的 example

（后面 LwIP 的基础例程主要直接使用或参考源码里的 example 即可）

我们打开之前下载好的 contrib-1.4.1 文件夹，如下图所示。



我们先讲解一下这个目录：

（1）**apps** 目录里实现了很多应用层协议。LwIP 源码包中也有 apps 目录，但源码包中 apps 目录下的应用程序全部用 RAW/Callback API 实现，属于内核代码的一部分。而此 apps 目录里的应用程序可以是由三种 API 中的任何一种实现的。读者可以把它看成是内核源码所提供的应用程序的一个补充。

（2）**ports** 目录里是一些移植文件，它可以帮助我们将 LwIP 移植到某个具体的操作系统中。目前这个目录所提供的移植文件，只支持 FreeRTOS、UNIX、Win32。我们会在后续的章节中讲解如何移植 LwIP。

2.6 LwIP 的三种编程接口

LwIP 提供了三种编程接口，分别为 RAW/Callback API、NETCONN API、SOCKET API。它们的易用性从左到右依次提高，而执行效率从左到右依次降低，用户可以根据实际情况，平衡利弊，选择合适的 API 进行网络应用程序的开发。以下内容将分别介绍这三种 API。

2.6.1 RAW/Callback API

RAW/Callback API 是指内核回调型的 API，这在许多通信协议的 C 语言实现中都有所应用。对于从来没有接触过回调式编程的人来说，可能理解起来会比较困难，我们在后面的章节中会详细介绍它。

RAW/Callback API 是 LwIP 的一大特色，在没有操作系统支持的裸机环境中，只能使用这种 API 进行开发，同时这种 API 也可以用在操作系统环境中。这里先简要说明一下“回调”的概念。你新建了一个 TCP 或者 UDP 的连接，你想等它接收到数据以后去处理它们，这时你需要把处理该数据的操作封装成一个函数，然后将这个函数的指针注册到 LwIP 内核中。LwIP 内核会在需要的时候去检测该连接是否收到数据，如果收到了数据，内核会在第一时间调用注册的函数，这个过程被称为“回调”，这个注册函数被称为“回调函数”。这个回调函数中装着你想要的业务逻辑，在这个函数中，你可以自由地处理接收到的数据，也可以发送任何数据，也就是说，这个回调函数就是你的应用程序。到这里，我们可以发现，在回调编程中，LwIP 内核把数据交给应用程序的过程就只是一次简单的函数调用，这是非常节省时间和空间资源的。每一个回调函数实际上只是一个普通的 C 函数，这个函数在 TCP/IP 内核中被调用。每一个回调函数都作为一个参数传递给当前 TCP 或 UDP 连接。而且，为了能够保存程序的特定状态，可以向回调函数传递一个指定的状态，并且这个指定的状态是独立于 TCP/IP 协议栈的。

在有操作系统的环境中，如果使用 RAW/Callback API，用户的应用程序就以回调函数的形式成为了内核代码的一部分，用户应用程序和内核程序会处于同一个线程之中，这就省去了任务间通信和切换任务的开销了。

简单来说，RAW/Callback API 的优点有两个：

(1) 可以在没有操作系统的环境中使用。

(2) 在有操作系统的环境中使用它，对比另外两种 API，可以提高应用程序的效率、节省内存开销。

RAW/Callback API 的优点是显著的，但缺点也是显著的：

(1) 基于回调函数开发应用程序时的思维过程比较复杂。在后面与 RAW/CallbackAPI 相关的章节中可以看到，利用回调函数去实现复杂的业务逻辑时，会很麻烦，而且代码的可读性较差。

(2) 在操作系统环境中，应用程序代码与内核代码处于同一个线程，虽然能够节省任务间通信和切换任务的开销，但是相应地，应用程序的执行会制约内核程序的执行，不同的应用程序之间也会互相制约。在应用程序执行的过程中，内核程序将不可能得到运行，这会影响网络数据包的处理效率。如果应用程序占用的时间过长，而且碰巧这时又有大量的数据包到达，由于内核代码长期得不到执行，网卡接收缓存里的数据包就持续积累，到最后很可能因为满载而丢弃一些数据包，从而造成丢包的现象。

2.6.2 NETCONN API

在操作系统环境中，可以使用 NETCONN API 或者 Socket API 进行网络应用程序的开发。NETCONN API 是基于操作系统的 IPC 机制（即信号量和邮箱机制）实现的，它的设计将 LwIP 内核代码和网络应用程序分离成了独立的线程。如此一来，LwIP 内核线程就只负责数据包的 TCP/IP 封装和拆封，而不用进行数据的应用层处理，大大提高了系统对网络数据包的处理效率。

前面提到，使用 RAW/Callback API 会造成内核程序和网络应用程序、不同网络应用程序之间的相互制约，如果使用 NETCONN API 或者 Socket API，这种制约将不复存在。在操作系统环境中，LwIP 内核会被实现为一个独立的线程，名为 tcpip_thread，使用 NETCONN API 或者 Socket API 的应用程序处在不同的线程中，我们可以根据任务的重要性，分配不同的优先级给这些线程，从而保证重要任务的时效性，分配优先级的原则具体见下图。

线程	优先级
LwIP 内核线程 <code>tcpip_thread</code>	很高
重要的网络应用程序	高
不太重要而且处理数据比较耗时的网络应用程序	低

NETCONN API 使用了操作系统的 IPC 机制，对网络连接进行了抽象，用户可以像操作文件一样操作网络连接（打开/关闭、读/写数据）。但是 NETCONN API 并不如操作文件的 API 那样简单易用。举个例子，调用 `f_read` 函数读文件时，读到的数据会被放在一个用户指定的数组中，用户操作起来很方便，而 NETCONN API 的读数据 API，就没有那么人性化了。用户获得的不是一个数组，而是一个特殊的数据结构 `netbuf`，用户如果想使用好它，就需要对内核的 `pbuf` 和 `netbuf` 结构体有所了解，我们会在后续的章节中对它们进行讲解。NETCONN API 之所以采取这种不人性的设计，是为了避免数据包在内核程序和应用程序之间发生拷贝，从而降低程序运行效率。当然，用户如果不在意数据递交时的效率问题，也可以把 `netbuf` 中的数据取出来拷贝到一个数组中，然后去处理这个数组。

简单来说，NETCONN API 的优缺点是：

（1）相较于 RAW/Callback API，NETCONN API 简化了编程工作，使用户可以按照操作文件的方式来操作网络连接。但是，内核程序和网络应用程序之间的数据包传递，需要依靠操作系统的信号量和邮箱机制完成，这需要耗费更多的时间和内存，另外还要加上任务切换的时间开销，效率较低。

（2）相较于 Socket API，NETCONN API 避免了内核程序和网络应用程序之间的数据拷贝，提高了数据递交的效率。但是，NETCONN API 的易用性不如 Socket API 好，它需要用户对 LwIP 内核所使用数据结构有一定的了解。

2.6.3 SOCKET API

Socket，即套接字，它对网络连接进行了高级的抽象，使得用户可以像操作文件一样操作网络连接。它十分易用，许多网络开发人员最早接触的就是 Socket 编程，Socket 已经成为了网络编程的标准。在不同的系统中，运行着不同的 TCP/IP 协议，但是只要它实现了 Socket 的接口，那么用 Socket 编写的网络应用程序就能在其中运行。可见用 Socket 编写的网络应用程序具有很好的可移植性。

不同的系统有自己的一套 Socket 接口。Windows 系统中支持的是 WinSock，UNIX/Linux 系统中支持的是 BSD Socket，它们虽然风格不一致，但大同小异。LwIP 中的 Socket API 是 BSD Socket。但是 LwIP 并没有也没办法实现全部的 BSD Socket，如果开发人员想要移植 UNIX/Linux 系统中的网络应用程序到使用 LwIP 的系统中，就要注意这一点。

相较于 NETCONN API，Socket API 具有更好的易用性。使用 Socket API 编写的程序可读性好，便于维护，也便于移植到其它的系统中。Socket API 在内核程序和应用程序之间存在数据的拷贝，这会降低数据递交的效率。另外，LwIP 的 Socket API 是基于 NETCONN API 实现的，所以效率上相较前者要打个折扣。

第 3 章 LwIP 无操作系统移植

本章开始正式进入 LwIP 移植的学习，在前面的章节中，都是打基础的部分，俗话说“基础不牢，地动山摇”，我们只有在了解 LwIP 的时候才开始移植，这样子会更加加深我们的印象。

LwIP 的官方源代码中并没有提供对任何芯片的底层驱动移植程序，因此这部分程序需要由开发者自己完成，本章我们带领大家学习如何给 PZ-ENC28J60 以太网模块移植 LwIP。本章分为如下几部分内容：

3.1 PZ-ENC28J60 模块简介

3.2 LwIP 移植

3.3 硬件设计

3.4 软件设计

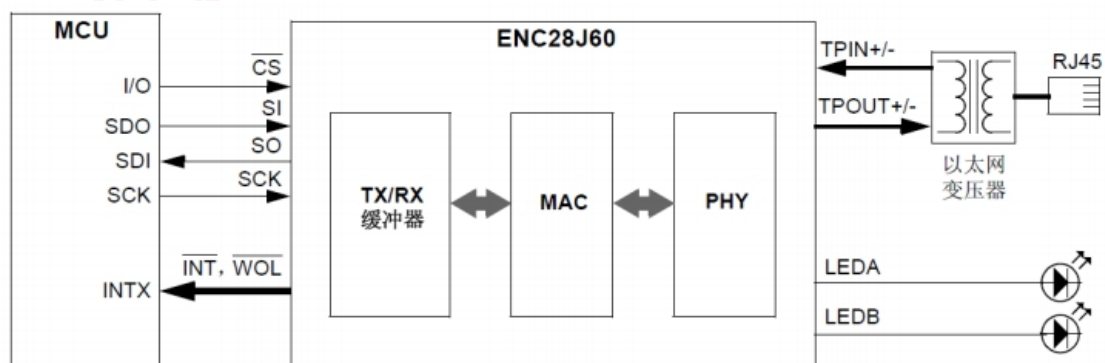
3.5 实验现象

3.1 PZ-ENC28J60 模块简介

ENC28J60 是带有行业标准串行外设接口（Serial Peripheral Interface, SPI）的独立以太网控制器。它可作为任何配备有 SPI 的 controllers 的以太网接口。ENC28J60 符合 IEEE 802.3 的全部规范，采用了一系列包过滤机制以对传入数据包进行限制。它还提供了一个内部 DMA 模块，以实现快速数据吞吐和硬件支持的 IP 校验和计算。与主控制器的通信通过两个中断引脚和 SPI 实现，数据传输速率高达 10 Mb/s。两个专用的引脚用于连接 LED，进行网络活动状态指示。ENC28J60 的主要特点如下：

- （1）兼容 IEEE802.3 协议的以太网控制器
- （2）集成 MAC 和 10 BASE-T 物理层
- （3）支持全双工和半双工模式
- （4）数据冲突时可编程自动重发
- （5）SPI 接口速度可达 10Mbps
- （6）8K 数据接收和发送双端口 RAM
- （7）提供快速数据移动的内部 DMA 控制器
- （8）可配置的接收和发送缓冲区大小
- （9）两个可编程 LED 输出
- （10）带 7 个中断源的两个中断引脚
- （11）TTL 电平输入
- （12）提供多种封装：SOIC/SSOP/SPDIP/QFN 等

ENC28J60 的典型应用电路如图所示：



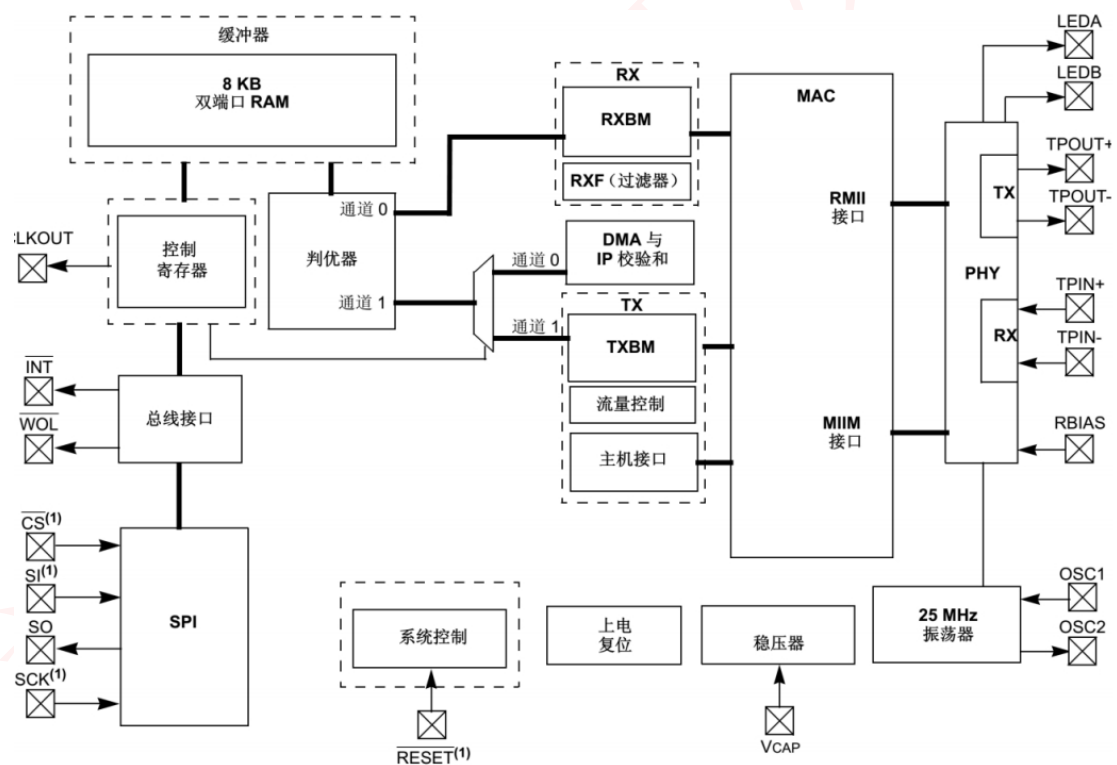
ENC28J60 由七个主要功能模块组成：

- 1) SPI 接口，充当主控制器和 ENC28J60 之间通信通道。

- 2) 控制寄存器，用于控制和监视 ENC28J60。
- 3) 双端口 RAM 缓冲器，用于接收和发送数据包。
- 4) 判优器，当 DMA、发送和接收模块发出请求时对 RAM 缓冲器的访问进行控制。
- 5) 总线接口，对通过 SPI 接收的数据和命令进行解析。
- 6) MAC(Medium Access Control)模块，实现符合 IEEE 802.3 标准的 MAC 逻辑。
- 7) PHY(物理层)模块，对双绞线上的模拟数据进行编码和译码。

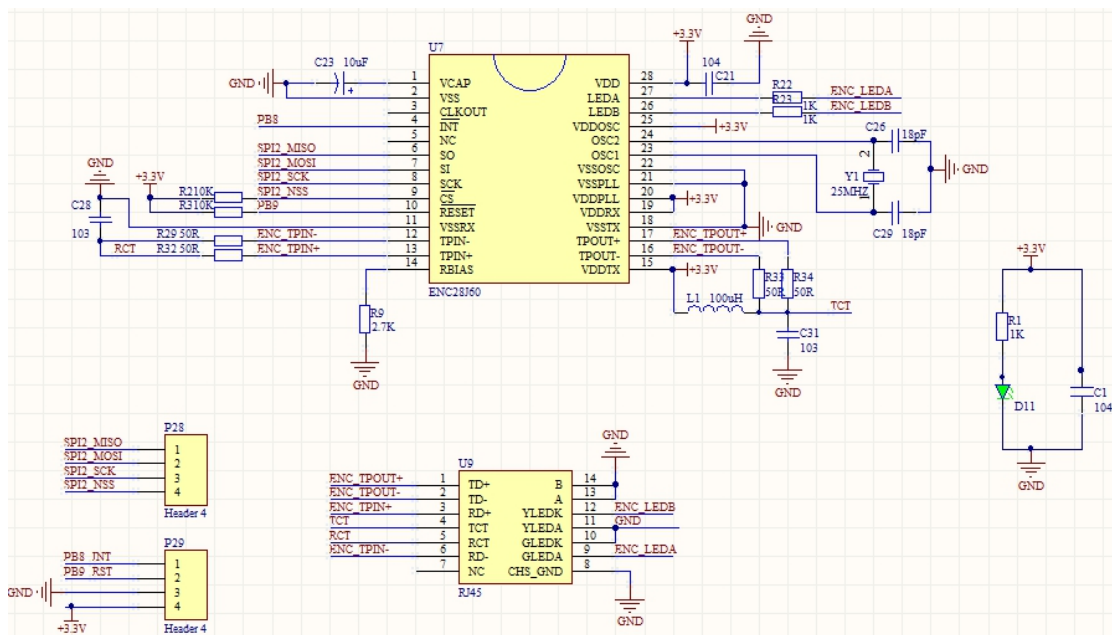
ENC28J60 还包括其他支持模块，诸如振荡器、片内稳压器、电平变换器（提供可以接受 5V 电压的 I/O 引脚）和系统控制逻辑。

ENC28J60 的功能框图如图所示：



PZ-ENC28J60 以太网模块采用 ENC28J60 作为主芯片，单芯片即可实现以太网接入，利用该模块，基本上只要是个单片机就可以实现以太网连接。

PZ-ENC28J60 以太网模块原理图如图所示：



PZ-ENC28J60 以太网模块外观图如图所示：



该模块通过一个 2*4 的排针与外部电路连接，这 8 个引脚分别是：3.3V、CS、GND、SCK、RST、MOSI、INT、MISO。其中 GND 和 3.3V 用于给模块供电，MISO/MOSI/SCK 用于 SPI 通信，CS 是片选信号，INT 为中断输出引脚，RST 为模块复位信号。

要将 PZ-ENC28J60 以太网模块连接到 STM32 开发板中，可使用杜邦线按照下列方式连接：（PZ-ENC28J60 以太网模块-->STM32 IO 口）

3.3V-->3.3V

GND-->GND

RST-->PG8

INT-->PG6

CS-->PG7

SCK-->PB13

MOSI-->PB15

MISO-->PB14

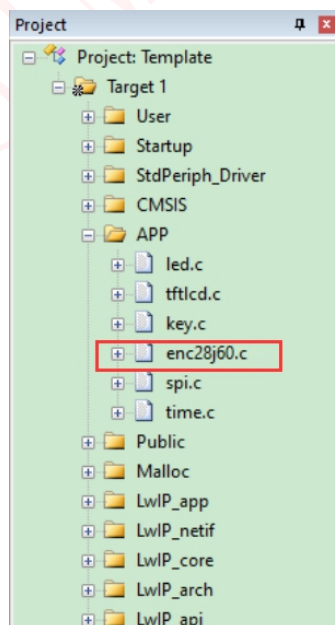
3.2 LwIP 移植

3.2.1 准备基础工程

首先我们需要一个基础工程作为模板，因为我们要使用到内存管理，所以就以基础实验中的“..\1--基础实验\36-内存管理实验”作为工程模板，并将文件夹重命名为“LwIP 无操作系统移植”；然后再将我们已经下载的 LwIP 源码拿过来，在我们资料内已经给大家提供了源码文件“..\LwIP 学习资料\lwip-1.4.1”。

3.2.2 添加网卡驱动程序

打开“..\1-LwIP 无操作系统移植”的 APP 文件夹，在里面有一个 ENC28J60 文件，在这个文件中有 enc28j60.c 和 enc28j60.h 这两个文件，这两个文件里面包含 ENC28J60 驱动程序，大家在移植的时候将 ENC28J60 文件拷贝到自己的工程 APP 文件中，并且将 enc28j60.c 添加到工程中，添加完成以后如下图所示：



在本实验中我们将 ENC28J60 模块接在 STM32 的 SPI2 接口上。在 enc28j60.h 中还有另一个结构体 dev_struct，结构体定义如下：

```
1. typedef struct{
```

```

2.      u8 enc28j60bank;    //ENC28J60 当前使用的控制寄存器块
3.      u32 NextPacketPtr;  //读指针
4.      u8 macaddr[6];      //MAC 地址
5.      }dev_struct;

```

结构体 dev_struct 用于保存 ENC28J60 的一些参数信息，比如当前使用的控制寄存器块、读指针地址、MAC 地址。下面介绍一下 enc28j60.c 文件，enc28j60.c 文件中有 16 个函数，如下图所示。

函数	说明
ENC28J60_Init ()	初始化 ENC28J60。
ENC28J60_Read_Op ()	读取 ENC28J60 控制寄存器(带操作码)。
ENC28J60_Write_Op ()	向 ENC28J60 寄存器写入数据(带操作码)。
ENC28J60_Read_Buf ()	读取 ENC28J60 接收缓存数据。
ENC28J60_Write_Buf()	向 ENC28J60 写发送缓存数据。
ENC28J60_Set_Bank ()	设置 ENC28J60 寄存器 Bank。
ENC28J60_Read ()	读取 ENC28J60 指定寄存器。
ENC28J60_Write ()	向 ENC28J60 指定寄存器写数据。
ENC28J60_PHY_Write ()	向 ENC28J60 的 PHY 寄存器写入数据。
ENC28J60_PHY_Read ()	读取 ENC28J60 的 PHY 寄存器。
ENC28J60_Get_EREVID ()	读取 ENC28J60 的版本号。
ENC28J60_Get_Duplex ()	获取 ENC28J60 双工状态。
ENC28J60_Packet_Send ()	通过 ENC28J60 发送数据包到网络。
ENC28J60_Packet_Receive ()	从网络获取一个数据包内容。
ENC28J60_ISRHandler ()	中断处理函数。
EXTIX_IRQHandler ()	外部中断服务函数(中断线视具体情况而定)。

接下来我们详细的介绍一下上图中几个重要的函数，首先来看一下 ENC28J60_Init() 函数。ENC28J60_Init() 函数代码如下。

```

1.      //初始化 ENC28J60
2.      //macaddr:MAC 地址
3.      //返回值:0,初始化成功;
4.      //      1,初始化失败;
5.      u8 ENC28J60_Init(void)
6.      {
7.          u8 version;
8.          u16 retry=0;
9.          u32 temp;
10.
11.          GPIO_InitTypeDef GPIO_InitStructure;
12.          SPI_InitTypeDef  SPI_InitStructure;
13.          EXTI_InitTypeDef EXTI_InitStructure;
14.          NVIC_InitTypeDef NVIC_InitStructure;
15.
16.          RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOG|RCC_APB2Periph_AFIO, ENABLE); //使
            能 PB,G 端口时钟
17.

```



```

18.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12;           //PB12 上拉 防止 W25X 的干扰
19.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;     //推挽输出
20.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
21.     GPIO_Init(GPIOB, &GPIO_InitStructure);               //初始化指定 IO
22.     GPIO_SetBits(GPIOB,GPIO_Pin_12);                     //上拉
23.
24.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7|GPIO_Pin_8;  //PG8 7 推挽
25.     GPIO_Init(GPIOG, &GPIO_InitStructure);
26.
27.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;             //中断引脚 PG6 上拉输入
28.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
29.     GPIO_Init(GPIOG, &GPIO_InitStructure);
30.
31.     //PG6 外部中断，中断线 6
32.     GPIO_EXTILineConfig(GPIO_PortSourceGPIOG,GPIO_PinSource6);
33.
34.     EXTI_InitStructure.EXTI_Line = EXTI_Line6;
35.     EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
36.     EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
37.     EXTI_InitStructure.EXTI_LineCmd = ENABLE;
38.     EXTI_Init(&EXTI_InitStructure);
39.
40.     EXTI_ClearITPendingBit(EXTI_Line6); //清除中断线 6 挂起标志位
41.
42.     NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;    //外部中断线 6
43.     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0; //抢占优先级
44.     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;     //子优先级
45.     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
46.     NVIC_Init(&NVIC_InitStructure);
47.
48.     GPIO_SetBits(GPIOG,GPIO_Pin_6|GPIO_Pin_7|GPIO_Pin_8); //PG6/7/8 上拉
49.     SPI2_Init();           //初始化 SPI
50.     SPI_Cmd(SPI2, DISABLE); // SPI 外设不使能
51.
52.     SPI_InitStructure.SPI_Direction = SPI_Direction_2Lines_FullDuplex; //SPI 设置为双线双向全双工
53.     SPI_InitStructure.SPI_Mode = SPI_Mode_Master;           //SPI 主机
54.     SPI_InitStructure.SPI_DataSize = SPI_DataSize_8b;       //发送接收 8 位帧结构
55.     SPI_InitStructure.SPI_CPOL = SPI_CPOL_Low;             //时钟悬空低
56.     SPI_InitStructure.SPI_CPHA = SPI_CPHA_1Edge;           //数据捕获于第 1 个时钟沿
57.     SPI_InitStructure.SPI_NSS = SPI_NSS_Soft;               //NSS 信号由软件控制
58.     SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_256; //定义波特率预分频的值:波特
    率预分频值为 256
59.     SPI_InitStructure.SPI_FirstBit = SPI_FirstBit_MSB;     //数据传输从 MSB 位开始
60.     SPI_InitStructure.SPI_CRCPolynomial = 7;                //CRC 值计算的多项式

```

```

61.     SPI_Init(SPI2, &SPI_InitStructure);           //根据 SPI_InitStruct 中指定的参数初始化外设 SPIx 寄存
器
62.     SPI_Cmd(SPI2, ENABLE); //使能 SPI 外设
63.
64.     SPI2_SetSpeed(SPI_BaudRatePrescaler_4); //SPI2 SCK 频率为 36M/4=9Mhz
65.     //初始化 MAC 地址
66.     temp=*(vu32*)(0x1FFF7E8); //获取 STM32 的唯一 ID 的前 24 位作为 MAC 地址后三字节
67.     enc28j60_dev.macaddr[0]=2;
68.     enc28j60_dev.macaddr[1]=0;
69.     enc28j60_dev.macaddr[2]=0;
70.     enc28j60_dev.macaddr[3]=(temp>>16)&0XFF; //低三字节用 STM32 的唯一 ID
71.     enc28j60_dev.macaddr[4]=(temp>>8)&0XFF;
72.     enc28j60_dev.macaddr[5]=temp&0XFF;
73.
74.     ENC28J60_RST=0;           //复位 ENC28J60
75.     delay_ms(10);
76.     ENC28J60_RST=1;           //复位结束
77.     delay_ms(10);
78.     ENC28J60_Write_Op(ENC28J60_SOFT_RESET,0,ENC28J60_SOFT_RESET); //软件复位
79.     while(!(ENC28J60_Read(ESTAT)&ESTAT_CLKRDY)&&retry<250) //等待时钟稳定
80.     {
81.         retry++;
82.         delay_ms(1);
83.     }
84.     if(retry>=250)return 1; //ENC28J60 初始化失败
85.     version=ENC28J60_Get_EREVID();           //获取 ENC28J60 的版本号
86.     printf("ENC28J60 Version:%d\r\n",version);
87.
88.     enc28j60_dev.NextPacketPtr=RXSTART_INIT;
89.     //接收缓冲器由一个硬件管理的循环 FIFO 缓冲器构成。
90.     //寄存器对 ERXSTH:ERXSTL 和 ERXNDH:ERXNDL 作
91.     //为指针，定义缓冲器的容量和其在存储器中的位置。
92.     //ERXST 和 ERXND 指向的字节均包含在 FIFO 缓冲器内。
93.     //当从以太网接口接收数据字节时，这些字节被顺序写入
94.     //接收缓冲器。但是当写入由 ERXND 指向的存储单元
95.     //后，硬件会自动将接收的下一字节写入由 ERXST 指向
96.     //的存储单元。因此接收硬件将不会写入 FIFO 以外的单
97.     //元。
98.     //设置接收起始字节
99.     ENC28J60_Write(ERXSTL,RXSTART_INIT&0XFF); //设置接收缓冲区起始地址低 8 位
100.    ENC28J60_Write(ERXSTH,RXSTART_INIT>>8); //设置接收缓冲区起始地址高 8 位
101.    //设置接收接收字节
102.    ENC28J60_Write(ERXNDL,RXSTOP_INIT&0XFF);
103.    ENC28J60_Write(ERXNDH,RXSTOP_INIT>>8);

```

```

104. //设置发送起始字节
105. ENC28J60_Write(ETXSTL,TXSTART_INIT&0XFF);
106. ENC28J60_Write(ETXSTH,TXSTART_INIT>>8);
107. //设置发送结束字节
108. ENC28J60_Write(ETXNDL,TXSTOP_INIT&0XFF);
109. ENC28J60_Write(ETXNDH,TXSTOP_INIT>>8);
110. //ERXWRPTH:ERXWRPTL 寄存器定义硬件向 FIFO 中
111. //的哪个位置写入其接收到的字节。 指针是只读的，在成
112. //功接收到一个数据包后，硬件会自动更新指针。 指针可
113. //用于判断 FIFO 内剩余空间的大小 8K-1500。
114. //设置接收读指针字节
115. ENC28J60_Write(ERXRDPTL,RXSTART_INIT&0XFF);
116. ENC28J60_Write(ERXRDPH,RXSTART_INIT>>8);
117. //接收过滤器
118. //UCEN: 单播过滤器使能位
119. //当 ANDOR = 1 时:
120. //1 = 目标地址与本地 MAC 地址不匹配的数据包将被丢弃
121. //0 = 禁止过滤器
122. //当 ANDOR = 0 时:
123. //1 = 目标地址与本地 MAC 地址匹配的数据包会被接受
124. //0 = 禁止过滤器
125. //CRCEN: 后过滤器 CRC 校验使能位
126. //1 = 所有 CRC 无效的数据包都将被丢弃
127. //0 = 不考虑 CRC 是否有效
128. //PMEN: 格式匹配过滤器使能位
129. //当 ANDOR = 1 时:
130. //1 = 数据包必须符合格式匹配条件，否则将被丢弃
131. //0 = 禁止过滤器
132. //当 ANDOR = 0 时:
133. //1 = 符合格式匹配条件的数据包将被接受
134. //0 = 禁止过滤器
135. //ENC28J60_Write(ERXFCON,ERXFCON_UCEN|ERXFCON_CRCEN|ERXFCON_PMEN);
136. ENC28J60_Write(ERXFCON,0);
137. ENC28J60_Write(EPMM0,0X3F);
138. ENC28J60_Write(EPMM1,0X30);
139. ENC28J60_Write(EPMCSSL,0Xf9);
140. ENC28J60_Write(EPMCSH,0Xf7);
141. //bit 0 MARXEN: MAC 接收使能位
142. //1 = 允许 MAC 接收数据包
143. //0 = 禁止数据包接收
144. //bit 3 TXPAUS: 暂停控制帧发送使能位
145. //1 = 允许 MAC 发送暂停控制帧（用于全双工模式下的流量控制）
146. //0 = 禁止暂停帧发送
147. //bit 2 RXPAUS: 暂停控制帧接收使能位

```

```

148. //1 = 当接收到暂停控制帧时，禁止发送（正常操作）
149. //0 = 忽略接收到的暂停控制帧
150. ENC28J60_Write(MACON1,MACON1_MARXEN|MACON1_TXPAUS|MACON1_RXPAUS);
151. //将 MACON2 中的 MARST 位清零，使 MAC 退出复位状态。
152. ENC28J60_Write(MACON2,0x00);
153. //bit 7-5 PADCFG2:PACDFG0: 自动填充和 CRC 配置位
154. //111 = 用 0 填充所有短帧至 64 字节长，并追加一个有效的 CRC
155. //110 = 不自动填充短帧
156. //101 = MAC 自动检测具有 8100h 类型字段的 VLAN 协议帧，并自动填充到 64 字节长。如果不
157. //是 VLAN 帧，则填充至 60 字节长。填充后还要追加一个有效的 CRC
158. //100 = 不自动填充短帧
159. //011 = 用 0 填充所有短帧至 64 字节长，并追加一个有效的 CRC
160. //010 = 不自动填充短帧
161. //001 = 用 0 填充所有短帧至 60 字节长，并追加一个有效的 CRC
162. //000 = 不自动填充短帧
163. //bit 4 TXCRCEN: 发送 CRC 使能位
164. //1 = 不管 PADCFG 如何，MAC 都会在发送帧的末尾追加一个有效的 CRC。 如果 PADCFG 规定要
165. //追加有效的 CRC，则必须将 TXCRCEN 置 1。
166. //0 = MAC 不会追加 CRC。 检查最后 4 个字节，如果不是有效的 CRC 则报告给发送状态向量。
167. //bit 0 FULDPX: MAC 全双工使能位
168. //1 = MAC 工作在全双工模式下。 PHCON1.PDPXMD 位必须置 1。
169. //0 = MAC 工作在半双工模式下。 PHCON1.PDPXMD 位必须清零。
170. ENC28J60_Write(MACON3,MACON3_PADCFG0|MACON3_TXCRCEN|MACON3_FRMLNEN|MACON3_FULDPX);
171. // 最大帧长度 1518
172. ENC28J60_Write(MAMXFLL,MAX_FRAMELEN&0XFF);
173. ENC28J60_Write(MAMXFLH,MAX_FRAMELEN>>8);
174. //配置背对背包间间隔寄存器 MABBIPG。当使用
175. //全双工模式时，大多数应用使用 15h 编程该寄存
176. //器，而使用半双工模式时则使用 12h 进行编程。
177. ENC28J60_Write(MABBIPG,0x15);
178. //配置非背对背包间间隔寄存器的低字节
179. //MAIPGL。 大多数应用使用 12h 编程该寄存器。
180. //如果使用半双工模式，应编程非背对背包间间隔
181. //寄存器的高字节 MAIPGH。 大多数应用使用 0Ch
182. //编程该寄存器。
183. ENC28J60_Write(MAIPGL,0x12);
184. ENC28J60_Write(MAIPGH,0x0C);
185. //设置 MAC 地址
186. ENC28J60_Write(MAADR5,enc28j60_dev.macaddr[0]);
187. ENC28J60_Write(MAADR4,enc28j60_dev.macaddr[1]);
188. ENC28J60_Write(MAADR3,enc28j60_dev.macaddr[2]);
189. ENC28J60_Write(MAADR2,enc28j60_dev.macaddr[3]);
190. ENC28J60_Write(MAADR1,enc28j60_dev.macaddr[4]);
191. ENC28J60_Write(MAADR0,enc28j60_dev.macaddr[5]);

```

```

192. //配置 PHY 为全双工 LEDB 为拉电流
193. ENC28J60_PHY_Write(PHCON1,PHCON1_PDPXMD);
194. //HDLDIS: PHY 半双工环回禁止位
195. //当 PHCON1.PDPXMD = 1 或 PHCON1.PLOOPBK = 1 时:
196. //此位可被忽略。
197. //当 PHCON1.PDPXMD = 0 且 PHCON1.PLOOPBK = 0 时:
198. //1 = 要发送的数据仅通过双绞线接口发出
199. //0 = 要发送的数据会环回到 MAC 并通过双绞线接口发出
200. ENC28J60_PHY_Write(PHCON2,PHCON2_HDLDIS);
201. //ECON1 寄存器
202. //寄存器 3-1 所示为 ECON1 寄存器,它用于控制
203. //ENC28J60 的主要功能。 ECON1 中包含接收使能、发
204. //送请求、DMA 控制和存储区选择位。
205. ENC28J60_Set_Bank(ECON1);
206. //EIE: 以太网中断允许寄存器
207. //bit 7 INTIE: 全局 INT 中断允许位
208. //1 = 允许中断事件驱动 INT 引脚
209. //0 = 禁止所有 INT 引脚的活动(引脚始终被驱动为高电平)
210. //bit 6 PKTIE: 接收数据包待处理中断允许位
211. //1 = 允许接收数据包待处理中断
212. //0 = 禁止接收数据包待处理中断
213. ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,EIE,EIE_INTIE|EIE_PKTIE|EIE_TXIE|EIE_TXERIE|EIE_RXERIE);
214. // enable packet reception
215. //bit 2 RXEN: 接收使能位
216. //1 = 通过当前过滤器的数据包将被写入接收缓冲器
217. //0 = 忽略所有接收的数据包
218. ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,ECON1,ECON1_RXEN);
219. printf("ENC28J60 Duplex:%s\r\n",ENC28J60_Get_Duplex()?"Full Duplex":"Half Duplex"); //获取双工方式
220. return 0;
221. }

```

函数 ENC28J60_Init() 为 ENC28J60 的初始化函数,该函数首先完成 SPI2 与 ENC28J60 连接的 IO 口,然后就是配置 ENC28J60 了。

ENC28J60_Read_Op() 和 ENC28J60_Write_Op() 分别为读取 ENC28J60 内部寄存器和向 ENC28J60 寄存器写入数据,但是这两个函数不能直接用来读取和向 ENC28J60 写数据,因为还需要确定要读取或写入的寄存器在哪个 Bank,这两个函数如下。

```

1. //读取 ENC28J60 控制寄存器(带操作码)
2. //op: 操作码
3. //addr: 寄存器地址/参数
4. //返回值: 读到的数据
5. u8 ENC28J60_Read_Op(u8 op,u8 addr)

```

```

6.  {
7.      u8 dat=0;
8.      ENC28J60_CS=0;
9.      dat=op|(addr&ADDR_MASK);
10.     SPI2_ReadWriteByte(dat);
11.     dat=SPI2_ReadWriteByte(0XFF);
12.     //如果是读取 MAC/MII 寄存器,则第二次读到的数据才是正确的,见手册 29 页
13.     if(addr&0x80)dat=SPI2_ReadWriteByte(0XFF);
14.     ENC28J60_CS=1;
15.     return dat;
16. }
17.
18. //向 ENC28J60 寄存器写入数据(带操作码)
19. //op: 操作码
20. //addr: 寄存器地址
21. //data: 参数
22. void ENC28J60_Write_Op(u8 op,u8 addr,u8 data)
23. {
24.     u8 dat=0;
25.     ENC28J60_CS=0;
26.     dat=op|(addr&ADDR_MASK);
27.     SPI2_ReadWriteByte(dat);
28.     SPI2_ReadWriteByte(data);
29.     ENC28J60_CS=1;
30. }

```

函数 ENC28J60_Read_Buf() 是从 ENC28J60 接收数据缓冲区中读取网络数据, 函数 ENC28J60_Write_Buf() 是向发送缓冲区中写入数据, 这两个函数如下:

```

1. //读取 ENC28J60 接收缓存数据
2. //len: 要读取的数据长度
3. //data: 输出数据缓存区(末尾自动添加结束符)
4. void ENC28J60_Read_Buf(u32 len,u8* data)
5. {
6.     ENC28J60_CS=0;
7.     SPI2_ReadWriteByte(ENC28J60_READ_BUF_MEM);
8.     while(len)
9.     {
10.         len--;
11.         *data=(u8)SPI2_ReadWriteByte(0);
12.         data++;
13.     }
14.     *data='\0';
15.     ENC28J60_CS=1;
16. }

```

```

17.
18. //向 ENC28J60 写发送缓存数据
19. //len:要写入的数据长度
20. //data:数据缓存区
21. void ENC28J60_Write_Buf(u32 len,u8* data)
22. {
23.     ENC28J60_CS=0;
24.     SPI2_ReadWriteByte(ENC28J60_WRITE_BUF_MEM);
25.     while(len)
26.     {
27.         len--;
28.         SPI2_ReadWriteByte(*data);
29.         data++;
30.     }
31.     ENC28J60_CS=1;
32. }

```

函数 ENC28J60_Read() 和 ENC28J60_Write() 才是真正的读取 ENC28J60 指定寄存器值和向 ENC28J60 指定寄存器写入数据的函数，函数代码如下：

```

1. //读取 ENC28J60 指定寄存器
2. //addr:寄存器地址
3. //返回值:读到的数据
4. u8 ENC28J60_Read(u8 addr)
5. {
6.     ENC28J60_Set_Bank(addr); //设置 BANK
7.     return ENC28J60_Read_Op(ENC28J60_READ_CTRL_REG,addr);
8. }
9.
10. //向 ENC28J60 指定寄存器写数据
11. //addr:寄存器地址
12. //data:要写入的数据
13. void ENC28J60_Write(u8 addr,u8 data)
14. {
15.     ENC28J60_Set_Bank(addr);
16.     ENC28J60_Write_Op(ENC28J60_WRITE_CTRL_REG,addr,data);
17. }

```

ENC28J60_PHY_Read() 和 ENC28J60_PHY_Write() 这两个函数是用来读写 ENC28J60 的 PHY 寄存器的，函数代码如下：

```

1. //向 ENC28J60 的 PHY 寄存器写入数据
2. //addr:寄存器地址
3. //data:要写入的数据
4. void ENC28J60_PHY_Write(u8 addr,u16 data)
5. {

```



```

6.      u16 retry=0;
7.      ENC28J60_Write(MIREGADR,addr);      //向 MIREGADR 写入 PHY 寄存器地址
8.      ENC28J60_Write(MIWRH,data);          //写入数据的低 8 字节
9.      ENC28J60_Write(MIWRH,data>>8);      //写入数据的高 8 字节
10.     while((ENC28J60_Read(MISTAT)&MISTAT_BUSY)&&retry<0xFFFF)retry++; //等待写 PHY 结束
11. }
12.
13. //读取 ENC28J60 的 PHY 寄存器
14. //addr:要读取的寄存器地址
15. //返回值:读取到的 PHY 的值
16. u16 ENC28J60_PHY_Read(u8 addr)
17. {
18.     u8 temp;
19.     u16 phyvalue,retry=0;
20.     temp=ENC28J60_Read(MICMD);
21.     ENC28J60_Write(MIREGADR,addr);
22.     ENC28J60_Write(MICMD,temp|MICMD_MIIRD); //开始读 PHY 寄存器
23.     while((ENC28J60_Read(MISTAT)&MISTAT_BUSY)&&retry<0xFFFF)retry++; //等待读 PHY 完成
24.     ENC28J60_Write(MICMD,temp&(~MICMD_MIIRD)); //读 PHY 完成
25.     phyvalue=ENC28J60_Read(MIRDL);          //读取低 8 位
26.     phyvalue|=(ENC28J60_Read(MIRDH)<<8);    //读取高 8 位
27.     return phyvalue;
28. }

```

函数 ENC28J60_Get_EREVID() 和 ENC28J60_Get_Duplex() 用来获取 ENC28J60 的芯片版本号和工作是的双工方式，很简单，这里就不讲解了。

函数 ENC28J60_Packet_Send() 是通过 ENC28J60 发送数据，代码如下：

```

1. //通过 ENC28J60 发送数据包到网络
2. //len:数据包大小
3. //packet:数据包
4. void ENC28J60_Packet_Send(u32 len,u8* packet)
5. {
6.     INTX_DISABLE();
7.     while(ENC28J60_Read(ECON1)&0X08); //正在发送数据，等待发送完成
8.     if(len>MAX_FRAMELEN)
9.     {
10.         ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,MACON3,MACON3_HFRMEN);
11.         ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,0X0E,PKTCTRL_POVERRIDE|PKTCTRL_PHUGREEN);
12.     }
13.     //设置发送缓冲区地址写指针入口
14.     ENC28J60_Write(EWRPTL,TXSTART_INIT&0XFF);
15.     ENC28J60_Write(EWRPTH,TXSTART_INIT>>8);
16.     //设置 TXND 指针，以对应给定的数据包大小
17.     ENC28J60_Write(ETXNDL,(TXSTART_INIT+len)&0XFF);

```

```

18.     ENC28J60_Write(ETXNDH,(TXSTART_INIT+len)>>8);
19.     //写每包控制字节(0x00表示使用macon3的设置)
20.     ENC28J60_Write_Op(ENC28J60_WRITE_BUF_MEM,0,0x00);
21.     //复制数据包到发送缓冲区
22.     ENC28J60_Write_Buf(len,packet);
23.     //发送数据到网络
24.     ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,ECON1,ECON1_TXRTS);
25.     //复位发送逻辑的问题。参见 Rev. B4 Silicon Errata point 12.
26.     if((ENC28J60_Read(EIR)&EIR_TXERIF))ENC28J60_Write_Op(ENC28J60_BIT_FIELD_CLR,ECON1,ECON1_TXRTS);
27.     INTX_ENABLE();
28. }

```

(1)、ENC28J60 在发送数据的时候不能被打断，包括中断也不行，这里调用函数 INTX_DISABLE() 用来关闭 STM32 的其他中断(不是所有的中断，此时硬 fault 和 NMI 中断依然有效)，函数 INTX_DISABLE() 的实现在 system.c 文件中。

(2)、在发送完成后就再次打开中断。

函数 ENC28J60_Packet_Receive() 用来接收网络数据。

```

1.     //从网络获取一个数据包内容
2.     //maxlen:数据包最大允许接收长度
3.     //packet:数据包缓存区
4.     //返回值:收到的数据包长度(字节)
5.     u32 ENC28J60_Packet_Receive(u32 maxlen,u8* packet)
6.     {
7.         u32 rxstat;
8.         u32 len;
9.         INTX_DISABLE();
10.        if(ENC28J60_Read(EPKTCNT)==0)return 0; //是否收到数据包?
11.        while((ENC28J60_Read(ESTAT)&0X04)); //接收忙，等待接收完成
12.        //设置接收缓冲器读指针
13.        ENC28J60_Write(ERDPTL,(enc28j60_dev.NextPacketPtr));
14.        ENC28J60_Write(ERDPTR,(enc28j60_dev.NextPacketPtr)>>8);
15.        // 读下一个包的指针
16.        enc28j60_dev.NextPacketPtr=ENC28J60_Read_Op(ENC28J60_READ_BUF_MEM,0);
17.        enc28j60_dev.NextPacketPtr|=ENC28J60_Read_Op(ENC28J60_READ_BUF_MEM,0)<<8;
18.        //读包的长度
19.        len=ENC28J60_Read_Op(ENC28J60_READ_BUF_MEM,0);
20.        len|=ENC28J60_Read_Op(ENC28J60_READ_BUF_MEM,0)<<8;
21.        len-=4; //去掉CRC计数
22.        //读取接收状态
23.        rxstat=ENC28J60_Read_Op(ENC28J60_READ_BUF_MEM,0);
24.        rxstat|=ENC28J60_Read_Op(ENC28J60_READ_BUF_MEM,0)<<8;
25.        //限制接收长度
26.        if (len>maxlen-1)len=maxlen-1;

```

```

27. //检查 CRC 和符号错误
28. // ERXFCN.CRCEN 为默认设置, 一般我们不需要检查.
29. if((rxstat&0x80)==0)len=0;//无效
30. else
31. {
32.     ENC28J60_Read_Buf(len,packet);//从接收缓冲器中复制数据包
33. }
34. //RX 读指针移动到下一个接收到的数据包的开始位置
35. //并释放我们刚才读出过的内存
36. ENC28J60_Write(ERXRDPTL,(enc28j60_dev.NextPacketPtr));
37. ENC28J60_Write(ERXRDPH,(enc28j60_dev.NextPacketPtr)>>8);
38. //递减数据包计数器标志我们已经得到了这个包
39. ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,ECON2,ECON2_PKTDEC);
40. INTX_ENABLE();
41. return(len);
42. }

```

函数 ENC28J60_ISRHandler() 为 DM9000 的中断处理函数, 注意! 不是中断服务函数, 函数 ENC28J60_ISRHandler() 会在 STM32 的中断服务函数中调用的。

```

1. //中断处理函数
2. void ENC28J60_ISRHandler(void)
3. {
4.     u8 status;
5.     u8 packetnum;
6.     u16 temp;
7.     ENC28J60_Write_Op(ENC28J60_BIT_FIELD_CLR,EIE,EIE_INTIE); //关闭 ENC28J60 的全局中断 ①
8.     status=ENC28J60_Read(EIR); //读取以太网中断标志寄存器
9.     if(status&EIR_PKTIF) //接收到数据, 处理数据
10.    {
11.        ENC28J60_Write_Op(ENC28J60_BIT_FIELD_CLR,EIR,EIR_PKTIF); //清除 ENC28J60 的接收中断标志位
12.        lwip_pkt_handle(); //清除 ENC28J60 的接收中断标志位 ②
13.    }
14.    if(status&EIR_TXIF) //以太网发送中断
15.    {
16.        ENC28J60_Write_Op(ENC28J60_BIT_FIELD_CLR,EIR,EIR_TXIF); //清除 ENC28J60 的接收中断标志位
17.    }
18.    }
19.    if(status&EIR_RXERIF) //接收错误中断标志位
20.    {
21.        ENC28J60_Write_Op(ENC28J60_BIT_FIELD_CLR,EIR,EIR_RXERIF);
22.        packetnum=ENC28J60_Read(EPKTCNT);
23.        temp=ENC28J60_Read(ERXRDPH)<<8; //读取高字节
24.        temp|=ENC28J60_Read(ERXRDPTL); //读取低字节
25.        temp++;

```

```

26.     ENC28J60_Write(ERXRDPTL,temp&0XFF);    //先写入低字节
27.     ENC28J60_Write(ERXRDPTH,temp>>8);    //先写入低字节
28.     ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,ECON2,ECON2_PKTDEC);
29.     printf("接收错误! 接收到数据包个数:%d\r\n",packetnum);
30. }
31. if(status&EIR_TXERIF)    //发送错误中断标志位
32. {
33.     ENC28J60_Write_Op(ENC28J60_BIT_FIELD_CLR,EIR,EIR_TXERIF);
34.     ENC28J60_Write_Op(ENC28J60_BIT_FIELD_CLR,ESTAT,ESTAT_LATECOL|ESTAT_TXABRT);
35.     printf("发送错误!\r\n");
36. }
37.     ENC28J60_Write_Op(ENC28J60_BIT_FIELD_SET,EIE,EIE_INTIE);    //打开 ENC28J60 的全局中断 ③
38. }

```

(1)、进入中断以后先关闭 ENC28J60 的中断。

(2)、当发生 EIR_RKTIF 中断的时候说明接收到数据，调用函数 `lwip_pkt_handle()` 接收数据，`lwip_pkt_handle()` 函数会在稍后讲解。

(3)、退出中断服务函数的时候就打开 ENC28J60 的中断。

最后就是中断线 6 的中断服务函数，函数很简单，代码如下：

```

1.    //外部中断线 6 的中断服务函数
2.    void EXTI9_5_IRQHandler(void)
3.    {
4.        EXTI_ClearITPendingBit(EXTI_Line6); //清除中断线 6 挂起标志位
5.        while(ENC28J60_INT == 0)
6.        {
7.            ENC28J60_ISRHandler();
8.        }
9.    }

```

3.2.3 LWIP 数据包和网络接口管理

以下关于 LWIP 数据包管理和网络接口管理部分参考自《嵌入式网络那些事 LWIP 协议深度剖析与实战演练》作者朱升林。

(1) LWIP 数据包管理

LWIP 协议栈使用 `pbuf` 结构体来描述协议栈使用中的数据包，`pbuf` 结构体在 `pbuf.h` 中有定义，定义如下。

```

1.    struct pbuf
2.    {
3.        struct pbuf *next; //指向下一个 pbuf 结构体，可以构成链表
4.        void *payload; //指向该 pbuf 真正的数据区

```

```

5.      u16_t tot_len; //当前 pbuf 和链表中后面所有 pbuf 的数据长度，它们属于一个数据包
6.      u16_t len; //当前 pbuf 的数据长度
7.      u8_t type; //当前 pbuf 的类型
8.      u8_t flags; //状态为，保留
9.      u16_t ref; //该 pbuf 被引用的次数
10.    };

```

next: 指向下一个 pbuf 结构体，每个 pbuf 能存放的数据有限，如果应用有大量的数据的话我们就需要多个 pbuf 来存放，我们将同一个数据包的 pbuf 连接在一起形成一个链表，那么 next 字段就是实现这个链表的关键。

payload: 指向该 pbuf 的数据存储区的首地址，网络模块接收到数据，并将数据提交给 LWIP 的时候，就是将数据存储在 payload 指定的存储区中。同样在发送数据的时候将 payload 所指向的存储区数据转给网络模块去发送。

tot_len: 我们在接收或发送数据的时候数据会存放在 pbuf 链表中，tot_len 字段就表示当前 pbuf 和链表中以后所有 pbuf 总的数据长度。

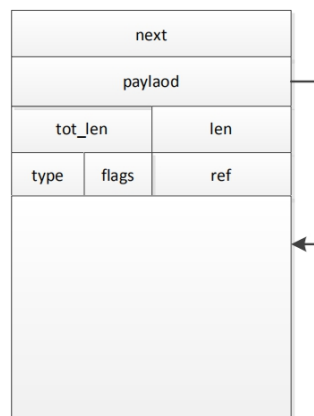
len: 当前 pbuf 总数据的长度

type: 当前 pbuf 类型，一共有四种：PBUF_RAM、PBUF_ROM、PBUF_REF 和 PBUF_POOL。

flag: 保留位

ref: 该 pbuf 被引用的次数，当还有其他指针指向这个 pbuf 的时候 ref 字段就加一。

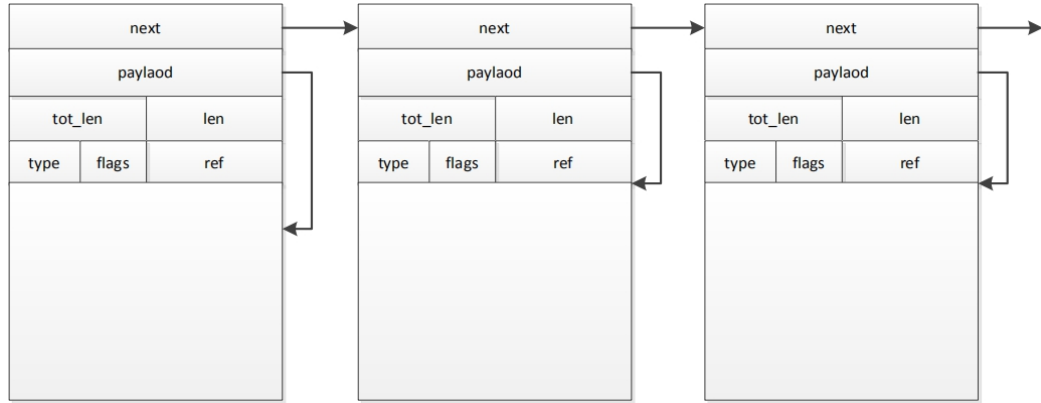
PBUF_RAM 类型的 pbuf 是通过内存堆分配得到的，PBUF_RAM 类型的 pbuf 如下图所示。



从图中可以看出 payload 并未指向数据区的起始地址，而是隔了一段区域，这段区域就是 offset，在里面通常存放 TCP 报文首部，IP 首部、以太网帧首

部等等。

PBUF_POOL 类型的 pbuf，PBUF_POOL 是通过内存池分配的到的，PBUF_POOL 类型的 pbuf 如下图所示。



从图中可以看出 pbuf 链表的第一个 pbuf 的 payload 未指向数据区的起始位置，原因同 PBUF_RAM 一样，用来存放一些首部的，pbuf 链表后面的 pbuf 结构体中 payload 字段就指向了数据区的起始位置。

(2) LWIP 网络接口管理

在 LWIP 中对于网络接口的描述是通过一个 netif 结构体完成的，netif 结构体在 netif.h 文件中有定义，定义如下，由于 netif 结构体比较大，这里只保留了一些比较重要的字段，具体的 netif 结构体定义请查阅 netif.h 文件。

```
1. struct netif
2. {
3.     struct netif *next; //指向下一个 netif 结构体
4.     ip_addr_t ip_addr; //网络接口 IP 地址
5.     ip_addr_t netmask; //子网掩码
6.     ip_addr_t gw; //默认网关
7.     netif_input_fn input; //IP 层接收数据函数
8.     netif_output_fn output; //IP 层发送数据包调用
9.     netif_linkoutput_fn linkoutput; //底层数据包发送
10.    void *state; //设备的状态信息
11.    u16_t mtu; //该网络接口最大允许传输的数据长度
12.    u8_t hwaddr_len; //物理地址长度
13.    u8_t hwaddr[NETIF_MAX_HWADDR_LEN]; //该网络接口的物理地址
14.    u8_t flags; //该网络接口的状态和属性
15.    char name[2]; //该网络接口的名字
16.    u8_t num; //该网络接口的编号
17. }
```

next: 该字段指向下一个 `netif` 类型的结构体，因为 LWIP 可以支持多个网络接口，当设备有多个网络接口的话 LWIP 就会把所有的 `netif` 结构体组成链表来管理这些网络接口。

ipaddr, netmask 和 gw 为分别为网络接口的 IP 地址、子网掩码和默认网关。

input: 此字段为一个函数，这个函数将网卡接收到的数据交给 IP 层。

output: 此字段为一个函数，当 IP 层向接口发送一个数据包时调用此函数。这个函数通常首先解析硬件地址，然后发送数据包。此字段我们一般使用 `etharp.c` 中的 `etharp_output()` 函数。

linkoutput: 此字段为一个函数，该函数被 ARP 模块调用，完成网络数据的发送。上面说的 `etharp_output` 函数将 IP 数据包封装成以太网数据帧以后就会调用 `linkoutput` 函数将数据发送出去。

state: 用来定义一些关于接口的信息，用户可以自行设置。

mtu: 网络接口所能传输的最大数据长度，一般设置为 1500

hwaddr_len 和 hwaddr 表示网络接口物理地址长度和具体的地址信息。

flags: 表示网络接口的状态和属性等信息，是很重要的字段，包括网卡功能使能，广播使能，ARP 使能等。

name: 网络接口的名字

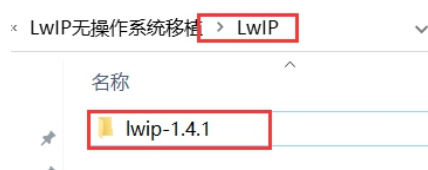
num: 网络接口的编号

以上我们只列出了 `netif` 结构体中几个比较重要的字段，我们对于网络接口的初始化就是给这些字段赋值。

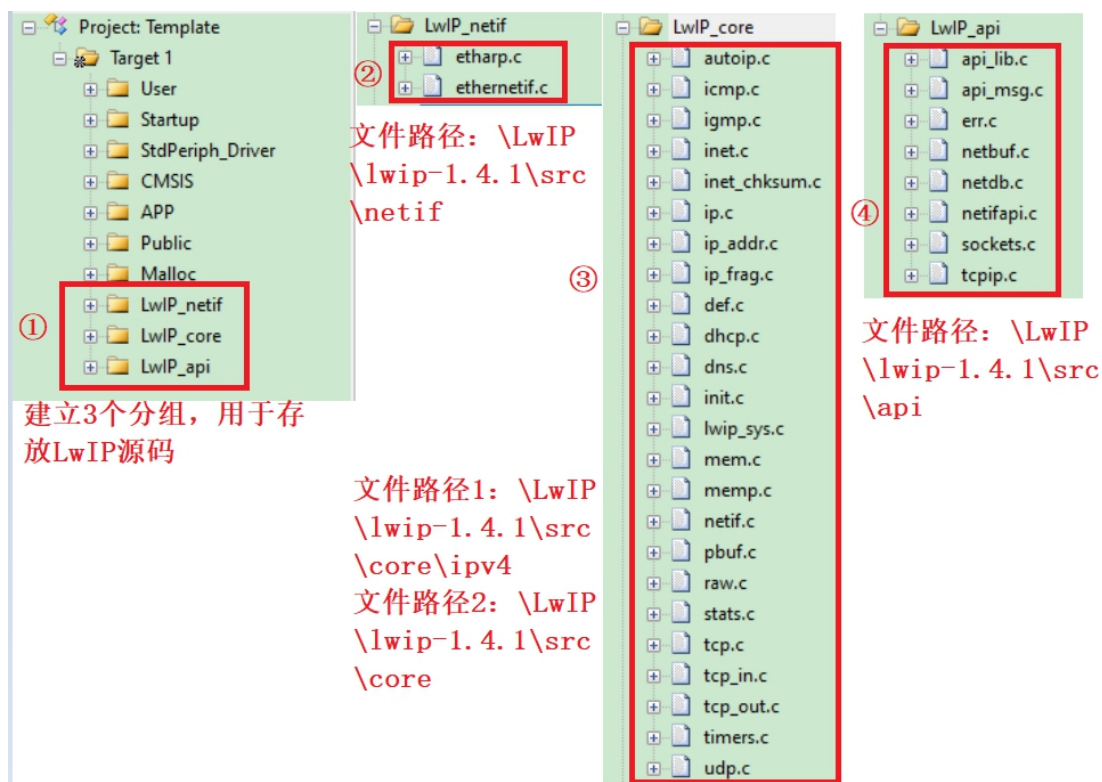
我们对于网络接口的初始化就是对 `netif` 结构体中各个字段的赋值。

3.2.4 添加 LWIP 源文件

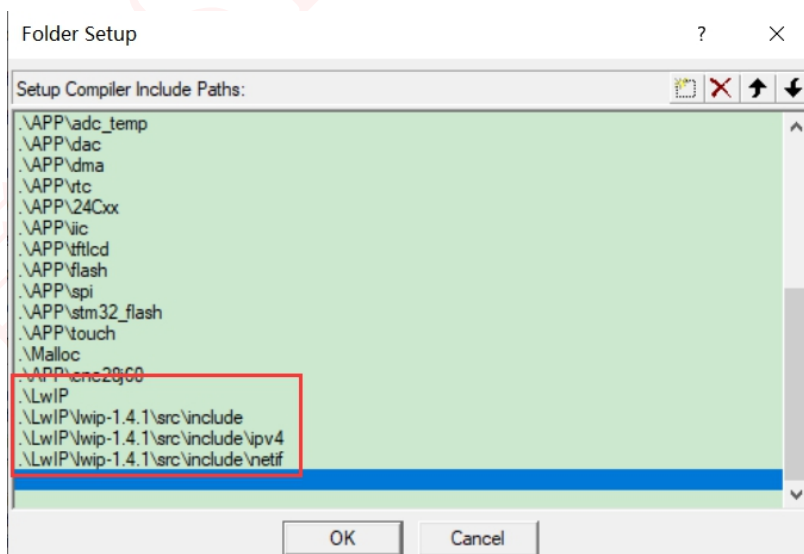
本节中我们将 LwIP 源码添加到我们的工程中，我们在工程目录中新建一个 LwIP 文件夹，在这个文件夹中我们放置所有关于 LwIP 的文件，新建完成后我们将 LwIP 的源码 `lwip-1.4.1` 拷贝到 LwIP 文件夹下，如下图所示。



将 LWIP 源文件拷贝到工程文件夹中以后我们就将里面的文件添加到工程中，按照下图所示添加到工程中。



我们将.c 文件添加到工程中以后，还需要添加 LWIP 源码中的头文件路径，头文件路径按照下图所示添加，这时如果我们编译的话会有很多错误提示，这个不用管。



3.2.5 添加中间文件

我们上面只是将 LWIP 源文件和以太网的驱动都添加到工程中，要将以太网驱动和 LWIP 连接起来还需要一些其他文件，而这些文件非常重要，我们下面就讲解如何添加这些文件。

(1) 添加 arch 文件

打开“..\1-LwIP 无操作系统移植”实验的 LwIP 文件夹可以发现有一个 arch 文件夹，将这个文件夹复制到自己工程中，在 arch 中有 5 个文件 cc.h、cpu.h、perf.h、sys_arch.h 和 sys_arch.c。根据 sys_arch.txt 中的描述，cc.h 主要完成了协议栈内部使用的数据类型的定义，如果使用操作系统的话还有临界代码区保护等等，cc.h 文件内容如下所示。

```
1.  #ifndef __CC_H__
2.  #define __CC_H__
3.  #include "cpu.h"
4.  #include "stdio.h"
5.  //定义与平台无关的数据类型
6.  typedef unsigned char u8_t; //无符号 8 位整数
7.  typedef signed char s8_t; //有符号 8 位整数
8.  typedef unsigned short u16_t; //无符号 16 位整数
9.  typedef signed short s16_t; //有符号 16 位整数
10. typedef unsigned long u32_t; //无符号 32 位整数
11. typedef signed long s32_t; //有符号 32 位整数
12. typedef u32_t mem_ptr_t; //内存地址型数据
13. typedef int sys_prot_t; //临界保护型数据
14. //使用操作系统时的临界区保护，这里以 UCOS II 为例
15. //当定义了 OS_CRITICAL_METHOD 时就说明使用了 UCOS II
16. #if OS_CRITICAL_METHOD == 1
17. #define SYS_ARCH_DECL_PROTECT(lev)
18. #define SYS_ARCH_PROTECT(lev) CPU_INT_DIS()
19. #define SYS_ARCH_UNPROTECT(lev) CPU_INT_EN()
20. #endif
21. #if OS_CRITICAL_METHOD == 3
22. #define SYS_ARCH_DECL_PROTECT(lev) u32_t lev
23. //UCOS II 中进入临界区,关中断
24. #define SYS_ARCH_PROTECT(lev) lev = OS_CPU_SR_Save()
25. //UCOS II 中退出 A 临界区,开中断
26. #define SYS_ARCH_UNPROTECT(lev) OS_CPU_SR_Restore(lev)
27. #endif
28. //根据不同的编译器定义一些符号
```

```

29.  #if defined (__ICCARM__)
30.  #define PACK_STRUCT_BEGIN
31.  #define PACK_STRUCT_STRUCT
32.  #define PACK_STRUCT_END
33.  #define PACK_STRUCT_FIELD(x) x
34.  #define PACK_STRUCT_USE_INCLUDES
35.  #elif defined (__CC_ARM)
36.  #define PACK_STRUCT_BEGIN __packed
37.  #define PACK_STRUCT_STRUCT
38.  #define PACK_STRUCT_END
39.  #define PACK_STRUCT_FIELD(x) x
40.  #elif defined (__GNUC__)
41.  #define PACK_STRUCT_BEGIN
42.  #define PACK_STRUCT_STRUCT __attribute__((packed))
43.  #define PACK_STRUCT_END
44.  #define PACK_STRUCT_FIELD(x) x
45.  #elif defined (__TASKING__)
46.  #define PACK_STRUCT_BEGIN
47.  #define PACK_STRUCT_STRUCT
48.  #define PACK_STRUCT_END
49.  #define PACK_STRUCT_FIELD(x) x
50.  #endif
51.  //LWIP 用 printf 调试时使用到的一些类型
52.  #define U16_F "4d"
53.  #define S16_F "4d"
54.  #define X16_F "4x"
55.  #define U32_F "8ld"
56.  #define S32_F "8ld"
57.  #define X32_F "8lx"
58.  //宏定义
59.  #ifndef LWIP_PLATFORM_ASSERT
60.  #define LWIP_PLATFORM_ASSERT(x) \
61.  do \
62.  { printf("Assertion \"%s\" failed at line %d in %s\r\n", x, __LINE__, __FILE__); \
63.  } while(0)
64.  #endif
65.  #ifndef LWIP_PLATFORM_DIAG
66.  #define LWIP_PLATFORM_DIAG(x) do {printf x;} while(0)
67.  #endif
68.  #endif

```

cpu.h 用来定义 CPU 的大小端模式，因为 STM32 是小端模式，因此这里定义 BYTE_ORDER 为小端模式，cpu.h 文件代码如下。

```

1.  #ifndef __CPU_H__

```

```

2.  #define __CPU_H__
3.  #define BYTE_ORDER LITTLE_ENDIAN //小端模式
4.  #endif

```

perf.h 是和系统测量与统计相关的文件，我们不使用任何的测量和统计，因此这个文件中的两个宏定义为空，代码如下所示。

```

1.  #ifndef __PERF_H__
2.  #define __PERF_H__
3.  #define PERF_START //空定义
4.  #define PERF_STOP(x) //空定义
5.  #endif

```

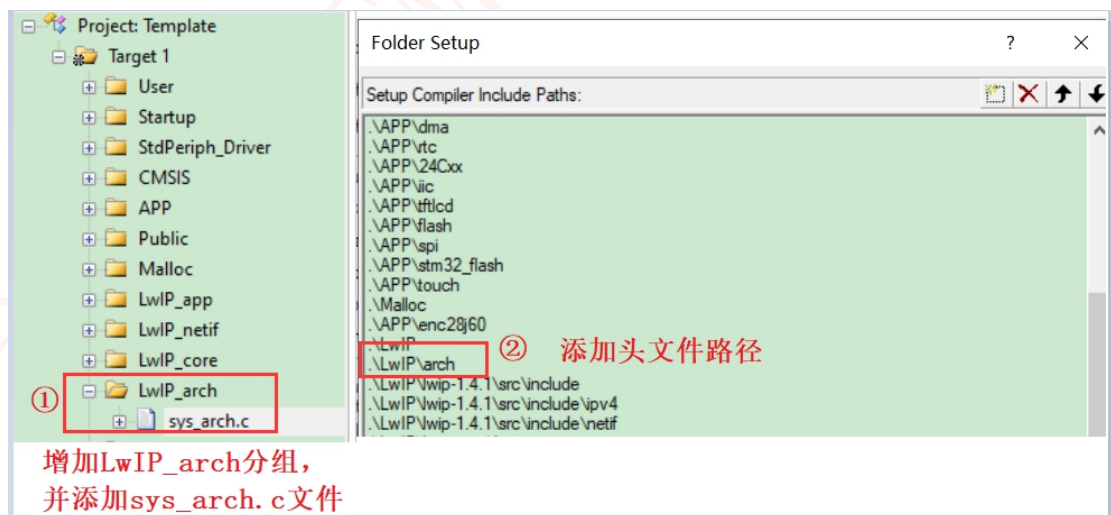
sys_arch.h 和 sys_arch.c 是在使用操作系统的时候才使用到的文件，在这里我们只是在 sys_arch.h 文件中简单的实现了获取时间的函数 sys_now()，代码如下所示，在 lwip_localtime 为一个全局变量，用来为 LWIP 提供时钟，这个变量我们会在下面的移植中介绍。

```

1.  u32_t sys_now(void)
2.  {
3.      return lwip_localtime;
4.  }

```

我们在工程中新建一个 LwIP_arch 分组，并将 sys_arch.c 文件添加到这个分组中并且添加相应的头文件路径，如下图所示。

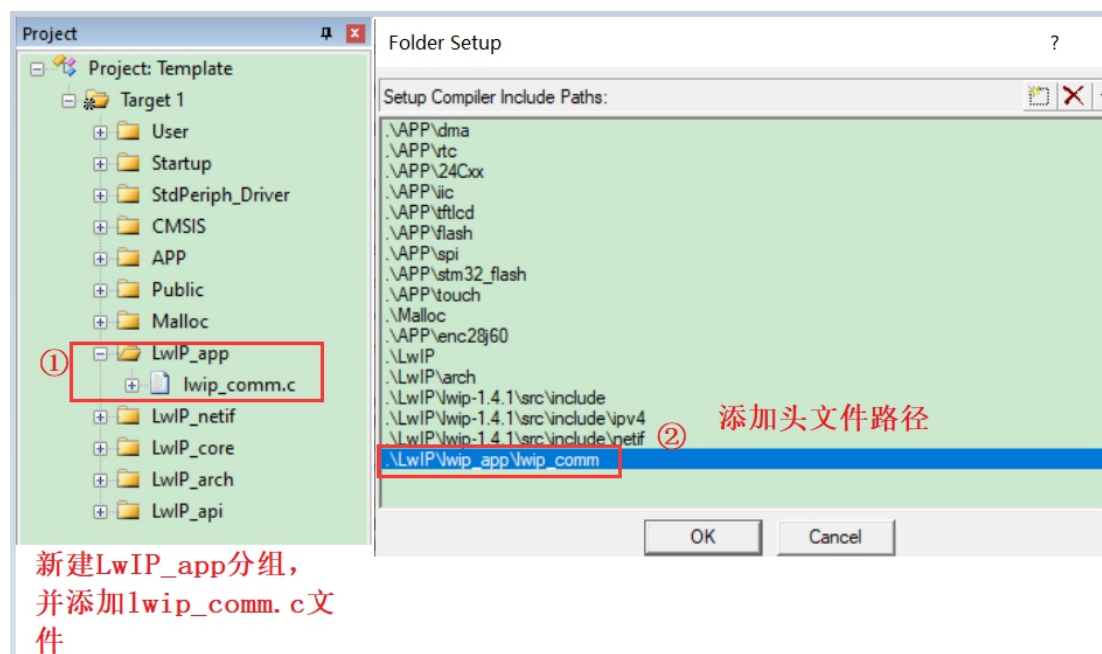


(2) 添加 LwIP 通用文件

打开“..\1-LwIP 无操作系统移植”实验的 LwIP 文件夹可以发现有一个 lwip_app 文件夹，将这个文件夹复制到自己工程中，lwip_app 文件夹用来放我们以后所有实验的代码。在 lwip_app 下有一个 lwip_comm 文件夹，这个文件中有 lwip_comm.c、lwip_comm.h 和 lwipopts.h

这三个文件，lwip_comm.c 和 lwip_comm.h 是将 LWIP 源码和前面的以太网驱动库结合起来的桥梁！这两个文件非常重要，这两个文件非 LwIP 源码文件，需自己编写，可直接使用我们配置好的。lwipopts.h 是用来裁剪和配置 LWIP 的文件，以后我们想要使用 LWIP 的什么功能的话就在这个文件中配置就行了。

同样的，我们在工程中新建一个 LwIP_app 分组，并将 lwip_comm.c 文件添加到这个分组中并且添加相应的头文件路径，如下图所示。



在 lwip_comm.h 中我们定义了一个重要的结构体__lwip_dev，这个结构体如下。

```

1. //lwip 控制结构体
2. typedef struct
3. {
4.     u8 mac[6]; //MAC 地址
5.     u8 remoteip[4]; //远端主机 IP 地址
6.     u8 ip[4]; //本机 IP 地址
7.     u8 netmask[4]; //子网掩码
8.     u8 gateway[4]; //默认网关的 IP 地址
9.
10.     vu8 dhcpstatus; //dhcp 状态
11.     //0, 未获取 DHCP 地址;
12.     //1, 进入 DHCP 获取状态
13.     //2, 成功获取 DHCP 地址
14.     //0xFF, 获取失败.
15. }__lwip_dev;

```

这个结构体有本机 MAC 地址、远端主机 IP 地址、本机 IP 地址、子网掩

码、默认网关和 DHCP 状态这几个成员变量。在 lwip_comm.c 中定义了一个 __lwip_dev 结构体类型变量 lwipdev，这是一个全局变量。

在 lwip_comm.c 中有以下 7 个函数：

```
1.  u8 lwip_comm_mem_malloc(void)
2.  void lwip_comm_mem_free(void)
3.  void lwip_comm_default_ip_set(__lwip_dev *lwipx)
4.  u8 lwip_comm_init(void)
5.  void lwip_pkt_handle(void)
6.  void lwip_periodic_handle()
7.  void lwip_dhcp_process_handle(void)
```

首先是 lwip_comm_mem_malloc() 函数，lwip_comm_mem_malloc() 函数完成了对 mem.c 和 memp.c 中内存堆 ram_heap 和内存池 memp_memory 的内存分配，函数代码如下。

```
1.  //lwip 中 mem 和 memp 的内存申请
2.  //返回值:0,成功;
3.  // 其他,失败
4.  u8 lwip_comm_mem_malloc(void)
5.  {
6.      u32 memsize;
7.      u32 ramheapsize;
8.      memsize=memp_get_memorysize(); //得到 memp_memory 数组大小
9.      memp_memory=mymalloc(SRAMIN,memsize); //为 memp_memory 申请内存
10.     ramheapsize=LWIP_MEM_ALIGN_SIZE(MEM_SIZE)+2*LWIP_MEM_ALIGN_SIZE(4*3)+MEM_ALIGNMENT;//得到 ram_heap 大小
11.     ram_heap=mymalloc(SRAMIN,ramheapsize); //为 ram_heap 申请内存
12.     if(!memp_memory||!ram_heap)//有申请失败的
13.     {
14.         lwip_comm_mem_free();
15.         return 1;
16.     }
17.     return 0;
18. }
```

lwip_comm_mem_free() 函数用来释放内存堆 ram_heap 和内存池 memp_memory 的内存，函数比较简单。

lwip_comm_default_ip_set() 函数用来设置默认地址，我们前面提过 __lwip_dev 结构体变量 lwipdev，lwip_comm_default_ip_set() 函数就是用来设置 lwipdev 的各个成员变量的。因为 MAC 地址要为全球唯一，因此这里 MAC 地址的前 3 个字节我们设置为 2、0、0。后三个字节我们取 STM32 的全球唯一

ID 的高三字节，当然在实际使用过程中也可以自行设置，只要保证在同一局域网中 MAC 地址不会重复就行。IP 地址、子网掩码、默认网关地址可以自行设置，这里我们设置 IP 地址为：192.168.1.30，子网掩码：255.255.255.0，默认网关：192.168.1.1，函数 `lwip_comm_default_ip_set()` 代码如下：

```
1. //lwip 默认 IP 设置
2. //lwipx:lwip 控制结构体指针
3. void lwip_comm_default_ip_set(__lwip_dev *lwipx)
4. {
5.     //默认远端 IP 为:192.168.1.100
6.     lwipx->remoteip[0]=192;
7.     lwipx->remoteip[1]=168;
8.     lwipx->remoteip[2]=1;
9.     lwipx->remoteip[3]=100;
10.    //MAC 地址设置(高三字节固定为:2.0.0,低三字节用 STM32 唯一 ID)
11.    lwipx->mac[0]=enc28j60_dev.macaddr[0];
12.    lwipx->mac[1]=enc28j60_dev.macaddr[1];
13.    lwipx->mac[2]=enc28j60_dev.macaddr[2];
14.    lwipx->mac[3]=enc28j60_dev.macaddr[3];
15.    lwipx->mac[4]=enc28j60_dev.macaddr[4];
16.    lwipx->mac[5]=enc28j60_dev.macaddr[5];
17.
18.    //默认本地 IP 为:192.168.1.30
19.    lwipx->ip[0]=192;
20.    lwipx->ip[1]=168;
21.    lwipx->ip[2]=1;
22.    lwipx->ip[3]=30;
23.    //默认子网掩码:255.255.255.0
24.    lwipx->netmask[0]=255;
25.    lwipx->netmask[1]=255;
26.    lwipx->netmask[2]=255;
27.    lwipx->netmask[3]=0;
28.    //默认网关:192.168.1.1
29.    lwipx->gateway[0]=192;
30.    lwipx->gateway[1]=168;
31.    lwipx->gateway[2]=1;
32.    lwipx->gateway[3]=1;
33.    lwipx->dhcpstatus=0;//没有 DHCP
34. }
```

接下来要介绍的 `lwip_comm_init` 函数是非常重要的一个函数，这个函数主要完成 LWIP 内核初始化、设置默认网卡并且打开指定的网卡，函数代码如下。

```
1. //LWIP 初始化(LWIP 启动的时候使用)
```

```

2. //返回值:0,成功
3. //      1,内存错误
4. //      2,DM9000 初始化失败
5. //      3,网卡添加失败.
6. u8 lwip_comm_init(void)
7. {
8.     struct netif *Netif_Init_Flag; //调用 netif_add()函数时的返回值,用于判断网络初始化是否成功
9.     struct ip_addr ipaddr; //ip 地址
10.    struct ip_addr netmask; //子网掩码
11.    struct ip_addr gw; //默认网关
12.    if(lwip_comm_mem_malloc())return 1; //内存申请失败
13.    if(ENC28J60_Init())return 2; //初始化 ENC28J60
14.    lwip_init(); //初始化 LWIP 内核
15.    lwip_comm_default_ip_set(&lwipdev); //设置默认 IP 等信息
16.
17.    #if LWIP_DHCP //使用动态 IP
18.        ipaddr.addr = 0;
19.        netmask.addr = 0;
20.        gw.addr = 0;
21.    #else //使用静态 IP
22.        IP4_ADDR(&ipaddr,lwipdev.ip[0],lwipdev.ip[1],lwipdev.ip[2],lwipdev.ip[3]);
23.        IP4_ADDR(&netmask,lwipdev.netmask[0],lwipdev.netmask[1],lwipdev.netmask[2],lwipdev.netmask[3]);
24.        IP4_ADDR(&gw,lwipdev.gateway[0],lwipdev.gateway[1],lwipdev.gateway[2],lwipdev.gateway[3]);
25.        printf("      网      卡      en      的      MAC      地      址
为:.....%d.%d.%d.%d\r\n",lwipdev.mac[0],lwipdev.mac[1],lwipdev.mac[2],lwipdev.mac[3],lwipdev.
mac[4],lwipdev.mac[5]);
26.        printf("      静      态      IP      地
址.....%d.%d.%d.%d\r\n",lwipdev.ip[0],lwipdev.ip[1],lwipdev.ip[2],lwipdev.ip[3]);
27.        printf("      子      网      掩
码.....%d.%d.%d.%d\r\n",lwipdev.netmask[0],lwipdev.netmask[1],lwipdev.netmask[2],lwipdev.
netmask[3]);
28.        printf("      默      认      网
关.....%d.%d.%d.%d\r\n",lwipdev.gateway[0],lwipdev.gateway[1],lwipdev.gateway[2],lwipdev.
gateway[3]);
29.    #endif
30.    Netif_Init_Flag=netif_add(&lwip_netif,&ipaddr,&netmask,&gw,NULL,0,netif_init,0,netif_input);//向网卡列
表中添加一个网口
31.
32.    #if LWIP_DHCP //如果使用 DHCP 的话
33.        lwipdev.dhcpstatus=0; //DHCP 标记为 0
34.        dhcp_start(&lwip_netif); //开启 DHCP 服务
35.    #endif
36.
37.    if(Netif_Init_Flag==NULL)return 3;//网卡添加失败

```

```

38.     else//网口添加成功后,设置 netif 为默认值,并且打开 netif 网口
39.     {
40.         netif_set_default(&lwip_netif); //设置 netif 为默认网口
41.         netif_set_up(&lwip_netif);      //打开 netif 网口
42.     }
43.     return 0;//操作 OK.
44. }

```

上面这个函数主要完成以下功能:

①调用 `lwip_comm_mem_malloc()` 函数完成前面提到的内存堆 `ram_heap` 和内存池 `memp_memory` 的内存分配。

②调用 `ENC28J60_Init()` 函数完成对 `ENC28J60` 的初始化, `ENC28J60_Init()` 函数在 `enc28j60.c` 文件中, 前面已经介绍过了。

③调用 `lwip_init` 函数完成 LWIP 的内核初始化, `lwip_init()` 通过调用各个模块的初始化函数来完成各个模块的初始化, 比如内存初始化函数、数据包结构初始化函数、网络接口初始化函数、IP 和 TCP 等的初始化函数, `lwip_init()` 在 `init.c` 文件中, 属于 LWIP 源码。

④调用 `ip_comm_default_ip_set()` 函数设置静态地址等信息。

⑤判断是否使用 DHCP, 如果使用 DHCP 的话就通过 DHCP 服务来获取 IP 地址、子网掩码和默认网关等信息, 如果使用静态 IP 地址的话就用 `ip_comm_default_ip_se()` 函数设置的地址信息。

⑥我们通过 `netif_add()` 函数来完成网卡的注册, `netif_add()` 在 `netif.c` 文件中, 此函数属于 LWIP 源码。 `netif_add()` 函数中的参数 `lwip_netif` 是我们定义的一个网络接口, 这个函数除了使用上面说的 IP 地址, 子网掩码和默认网关作为参数外, 还使用了两个函数地址作为参数: `ethernetif_init` 和 `ethernet_input`, 这两个函数地址会被赋值给 `netif` 结构体的相关字段, `ethernetif_init()` 在下一节会讲到, 这个函数由我们提供, `ethernet_input()` 函数在 `etharp.c` 文件中, 属于 LWIP 源码, 是 ARP 层数据包输入函数。

⑦如果使用 DHCP 的话就开启 DHCP 服务, 通过调用 `dhcp_start()` 函数开启 DHCP 服务。

⑧当网卡注册成功后使用 `netif_set_default()` 设置此网卡为默认网卡, 并且使用 `netif_set_up()` 函数打开此网卡。

`lwip_pkt_handle()` 函数来接收数据, `lwip_pkt_handle()` 函数代码如下。


```

1. //当接收到数据后调用
2. void lwip_pkt_handle(void)
3. {
4.     //从网络缓冲区中读取接收到的数据包并将其发送给 LWIP 处理
5.     ethernetif_input(&lwip_netif);
6. }

```

可以看出 `lwip_pkt_handle()` 函数其实只是对 `ethernetif_input()` 函数的简单封装，通过调用 `ethernetif_input()` 函数从指定的网络接口中接收数据，`ethernetif_input()` 函数是在 `ethernetif.c` 文件中定义的，我们在下一节会讲到。

LWIP 内核中有许多的周期性的定时器，相对应的定时处理函数也需要被周期性调用的，因为没有使用操作系统，所有需要我们自己使用定时器来实现。这里我们使用 STM32F103 定时器 3 提供这个系统时钟，定时器 3 定时周期为 10ms，定时器 3 的中断服务函数如下，`lwip_localtime` 是一个全局变量，在 `lwip_comm.c` 文件中有定义。我们新建 `time.c` 文件放关于定时器 3 的程序，然后将 `time.c` 添加到 APP 组下面，并且添加相应的头文件路径。

```

1. //定时器 3 中断服务函数
2. void TIM3_IRQHandler(void)
3. {
4.     if(TIM_GetITStatus(TIM3,TIM_IT_Update)==SET) //溢出中断
5.     {
6.         lwip_localtime +=10; //加 10
7.     }
8.     TIM_ClearITPendingBit(TIM3,TIM_IT_Update); //清除中断标志位
9. }

```

我们上面说了 LWIP 内核中有许多的周期性定时器，如 TCP 定时器、ARP 定时器、IP 如果使用 DHCP 的话还有 DHCP 定时器等等，我们可以将这些定时器封装在一个函数里面，然后周期性这个函数就行了，LWIP 协议栈要求的是每 250ms 处理一次 TCP 定时器，每 5S 处理一次 ARP 定时器，每 500ms 处理一次 DHCP 精细处理定时器，每 60s 执行一次 DHCP 的粗糙处理。我们就根据这个要求来编写 `lwip_periodic_handle()` 函数，函数代码如下，最后我们只要周期性的调用 `lwip_periodic_handle()` 函数就可以完成对 LWIP 内核的定时处理函数的周期性调用。

```

1. //LWIP 轮询任务
2. void lwip_periodic_handle()

```

```

3.  {
4.
5.  #if LWIP_TCP
6.      //每 250ms 调用一次 tcp_tmr()函数
7.      if (lwip_localtime - TCPTimer >= TCP_TMR_INTERVAL)
8.      {
9.          TCPTimer = lwip_localtime;
10.         tcp_tmr();
11.     }
12. #endif
13.     //ARP 每 5s 周期性调用一次
14.     if ((lwip_localtime - ARPTimer) >= ARP_TMR_INTERVAL)
15.     {
16.         ARPTimer = lwip_localtime;
17.         etharp_tmr();
18.     }
19.
20. #if LWIP_DHCP //如果使用 DHCP 的话
21.     //每 500ms 调用一次 dhcp_fine_tmr()
22.     if (lwip_localtime - DHCPfineTimer >= DHCP_FINE_TIMER_MSECS)
23.     {
24.         DHCPfineTimer = lwip_localtime;
25.         dhcp_fine_tmr();
26.         if ((lwipdev.dhcpstatus != 2)&&(lwipdev.dhcpstatus != 0XFF))
27.         {
28.             lwip_dhcp_process_handle(); //DHCP 处理
29.         }
30.     }
31.
32.     //每 60s 执行一次 DHCP 粗糙处理
33.     if (lwip_localtime - DHCPcoarseTimer >= DHCP_COARSE_TIMER_MSECS)
34.     {
35.         DHCPcoarseTimer = lwip_localtime;
36.         dhcp_coarse_tmr();
37.     }
38. #endif
39. }

```

lwip_dhcp_process_handle() 函数为 DHCP 处理函数，函数代码如下。

```

1.  //如果使能了 DHCP
2.  #if LWIP_DHCP
3.
4.  //DHCP 处理任务
5.  void lwip_dhcp_process_handle(void)
6.  {

```

```

7.      u32 ip=0,netmask=0,gw=0;
8.      switch(lwipdev.dhcpstatus)
9.      {
10.         case 0:      //开启 DHCP
11.             dhcp_start(&lwip_netif);
12.             lwipdev.dhcpstatus = 1;      //等待通过 DHCP 获取到的地址
13.             printf("正在查找 DHCP 服务器,请稍等.....\r\n");
14.             break;
15.         case 1:      //等待获取到 IP 地址
16.         {
17.             ip=lwip_netif.ip_addr.addr;      //读取新 IP 地址
18.             netmask=lwip_netif.netmask.addr; //读取子网掩码
19.             gw=lwip_netif.gw.addr;           //读取默认网关
20.             printf("正在获取 IP 地址\r\n");
21.             if(ip!=0)      //正确获取到 IP 地址的时候
22.             {
23.                 lwipdev.dhcpstatus=2;      //DHCP 成功
24.                 printf("网 卡 的 MAC 地 址
为:.....%d.%d.%d.%d\r\n",lwipdev.mac[0],lwipdev.mac[1],lwipdev.mac[2],lwipdev.mac[3],lwipdev.
mac[4],lwipdev.mac[5]);
25.                 //解析出通过 DHCP 获取到的 IP 地址
26.                 lwipdev.ip[3]=(uint8_t)(ip>>24);
27.                 lwipdev.ip[2]=(uint8_t)(ip>>16);
28.                 lwipdev.ip[1]=(uint8_t)(ip>>8);
29.                 lwipdev.ip[0]=(uint8_t)(ip);
30.                 printf("通 过 DHCP 获 取 到 IP 地
址.....%d.%d.%d.%d\r\n",lwipdev.ip[0],lwipdev.ip[1],lwipdev.ip[2],lwipdev.ip[3]);
31.                 //解析通过 DHCP 获取到的子网掩码地址
32.                 lwipdev.netmask[3]=(uint8_t)(netmask>>24);
33.                 lwipdev.netmask[2]=(uint8_t)(netmask>>16);
34.                 lwipdev.netmask[1]=(uint8_t)(netmask>>8);
35.                 lwipdev.netmask[0]=(uint8_t)(netmask);
36.                 printf("通 过 DHCP 获 取 到 子 网 掩
码 .....%d.%d.%d.%d\r\n",lwipdev.netmask[0],lwipdev.netmask[1],lwipdev.netmask[2],lwipdev.netmask[3]);
37.                 //解析出通过 DHCP 获取到的默认网关
38.                 lwipdev.gateway[3]=(uint8_t)(gw>>24);
39.                 lwipdev.gateway[2]=(uint8_t)(gw>>16);
40.                 lwipdev.gateway[1]=(uint8_t)(gw>>8);
41.                 lwipdev.gateway[0]=(uint8_t)(gw);
42.                 printf("通 过 DHCP 获 取 到 的 默 认 网
关.....%d.%d.%d.%d\r\n",lwipdev.gateway[0],lwipdev.gateway[1],lwipdev.gateway[2],lwipdev.gateway[3]);
43.             }else if(lwip_netif.dhcp->tries>LWIP_MAX_DHCP_TRIES) //通过 DHCP 服务获取 IP 地址失败,且超过最大尝试
次数

```

```

44.         {
45.             lwipdev.dhcpstatus=0XFF; //DHCP 超时失败.
46.             //使用静态 IP 地址
47.             IP4_ADDR(&(lwip_netif.ip_addr),lwipdev.ip[0],lwipdev.ip[1],lwipdev.ip[2],lwipdev.ip[3]);
48.             IP4_ADDR(&(lwip_netif.netmask),lwipdev.netmask[0],lwipdev.netmask[1],lwipdev.netmask[2],lw
ipdev.netmask[3]);
49.             IP4_ADDR(&(lwip_netif.gw),lwipdev.gateway[0],lwipdev.gateway[1],lwipdev.gateway[2],lwipdev.
gateway[3]);
50.             printf("DHCP 服务超时,使用静态 IP 地址!\r\n");
51.             printf("网 卡 的 MAC 地 址
为:.....%d.%d.%d.%d\r\n",lwipdev.mac[0],lwipdev.mac[1],lwipdev.mac[2],lwipdev.mac[3],lwipdev.
mac[4],lwipdev.mac[5]);
52.             printf("静 态 IP 地
址.....%d.%d.%d.%d\r\n",lwipdev.ip[0],lwipdev.ip[1],lwipdev.ip[2],lwipdev.ip[3]);
53.             printf("子 网 掩
码.....%d.%d.%d.%d\r\n",lwipdev.netmask[0],lwipdev.netmask[1],lwipdev.netmask[2],lwipdev.
netmask[3]);
54.             printf("默 认 网
关.....%d.%d.%d.%d\r\n",lwipdev.gateway[0],lwipdev.gateway[1],lwipdev.gateway[2],lwipdev.
gateway[3]);
55.         }
56.         delay_ms(250);
57.     }
58.     break;
59.     default : break;
60. }
61. }
62. #endif

```

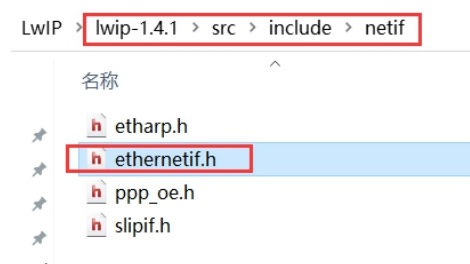
我们通过判断 lwipdev 结构体的 dhcpstatus 字段判断是否使用 DHCP 服务,当 dhcpstatus=0 时表示开启 DHCP,我们调用 dhcp_start() 函数开启相应网络接口的 DHCP 服务 dhcp_start() 函数由 LWIP 源码提供。开启 DHCP 以后让 dhcpstatus=1,表示开始进行 DHCP,等待 DHCP 完成。当 DHCP 完成以后让 dhcpstatus=2,表示 DHCP 成功。但是当 DHCP 重试次数大于 LWIP_MAX_DHCP_TRIES 时,意味着 DHCP 失败,这是 dhcpstatus=0XFF,表示 DHCP 失败,并且使用静态 IP 地址。

(3) 添加 ethernetif.h 文件

打开“[..\1-LwIP 无操作系统移植](#)”实验,

LwIP->lwipl.4.1->src->include->netif 中我们会发现有个 ethernetif.h 文件,这个文件在 LWIP 源码中是不存在的,将 ethernetif.h 文件复制到自己工

程的相应位置，如下图所示。



3.2.6 LWIP 源码修改

在上面添加中间文件完成后，我们还有修改一下 LWIP 的源码，中间文件连接 LWIP 底层驱动和 LWIP，这样或多或少会对 LWIP 的源码做出一点小的改动，下面我们就讲解如何修改 LWIP 源码。

(1) 修改 LWIP 源文件名字

我们按路径：lwip-1.4.1->src->core 可以发现 core 文件下有一个 sys.c 文件，按路径 lwip-1.4.1->src->include->lwip 可以发现有一个 sys.h 文件。sys.c 和 sys.h 这两个文件和我们文件中的 sys.c 和 sys.h 重名，因此我们将 LWIP 中 sys.c 和 sys.h 改为 lwip_sys.c 和 lwip_sys.h，然后在工程中将 LWIP 源码里面的#include “sys.h” 代码也要改掉，更改为#include “lwip_sys.h”。

(2) 修改 ethernetif.c 文件

ethernetif.c 的文件路径为：LWIP->lwip1.4.1->src->netif。用我们“[.. \1-LwIP 无操作系统移植](#)”实验中的 ethernetif.c 文件替代 LWIP 源码中的这个文件，在这个文件中有 5 个函数。

```
1. static err_t low_level_init(struct netif *netif)
2. static err_t low_level_output(struct netif *netif, struct pbuf *p)
3. static struct pbuf * low_level_input(struct netif *netif)
4. err_t ethernetif_input(struct netif *netif)
5. err_t ethernetif_init(struct netif *netif)
```

这 5 个函数是移植 LWIP 的重点，其中前 3 个与网卡密切相关，low_level_init 函数主要完成网卡的复位、协议栈网络接口管理结构体 netif 中相关字段的初始化，low_level_init 函数如下：

```
1. //由 ethernetif_init()调用用于初始化硬件
2. //netif:网卡结构体指针
```

```

3. //返回值:ERR_OK,正常
4. //      其他,失败
5. static err_t low_level_init(struct netif *netif)
6. {
7.     netif->hwaddr_len = ETHARP_HWADDR_LEN; //设置 MAC 地址长度,为 6 个字节
8.     //初始化 MAC 地址,设置什么地址由用户自己设置,但是不能与网络中其他设备 MAC 地址重复
9.     netif->hwaddr[0]=lwipdev.mac[0];
10.    netif->hwaddr[1]=lwipdev.mac[1];
11.    netif->hwaddr[2]=lwipdev.mac[2];
12.    netif->hwaddr[3]=lwipdev.mac[3];
13.    netif->hwaddr[4]=lwipdev.mac[4];
14.    netif->hwaddr[5]=lwipdev.mac[5];
15.    netif->mtu=1500; //最大允许传输单元,允许该网卡广播和 ARP 功能
16.    netif->flags = NETIF_FLAG_BROADCAST|NETIF_FLAG_ETHARP|NETIF_FLAG_LINK_UP;
17.    return ERR_OK;
18. }

```

low_level_output 函数用于发送数据,将 LWIP 协议栈准备好的数据通过网卡发送出去,先将 pbuf 结构的数据放到 buffer 中,然后调用函数 ENC28J60_Packet_Send() 发送数据,low_level_output 函数代码如下:

```

1. //用于发送数据包的最底层函数(lwip 通过 netif->linkoutput 指向该函数)
2. //netif:网卡结构体指针
3. //p:pbuf 数据结构体指针
4. //返回值:ERR_OK,发送正常
5. //      ERR_MEM,发送失败
6. static err_t low_level_output(struct netif *netif,struct pbuf *p)
7. {
8.     struct pbuf *q;
9.     int l=0;
10.    u8 *buffer;
11.    buffer=mymalloc(SRAMIN,p->tot_len); //申请内存
12.    if(buffer==NULL)printf("发送数据缓冲区内存申请失败\r\n");
13.    for(q=p;q!=NULL;q=q->next)
14.    {
15.        memcpy((u8_t*)&buffer[l], q->payload, q->len);
16.        l=l+q->len;
17.    }
18.    ENC28J60_Packet_Send(p->tot_len,buffer);
19.    myfree(SRAMIN,buffer); //释放内存
20.    return ERR_OK;
21. }

```

low_level_input 函数是从网卡中提取数据,并将数据封装在 pbuf 结构体中供 LWIP 使用,函数 low_level_input() 首先调用函数

ENC28J60_Packet_Receive() 接收网络数据，然后将数据打包成 pbuf 结构，然后将这个 pbuf 返回，low_level_input 函数就是完成这个功能的，代码如下所示。

```
1.  //用于接收数据包的最低层函数
2.  //netif:网卡结构体指针
3.  //返回值:pbuf 数据结构体指针
4.  static struct pbuf * low_level_input(struct netif *netif)
5.  {
6.      struct pbuf *p,*q;
7.      u32 len;
8.      u8 *buffer;
9.      int l=0;
10.
11.      p=NULL;
12.      buffer=mymalloc(SRAMIN,1600);          //申请内存
13.      if(buffer!=NULL)len=ENC28J60_Packet_Receive(MAX_FRAMELEN,buffer); //接收数据
14.      else
15.      {
16.          printf("接收数据缓冲区内内存申请失败\r\n");
17.          return p;
18.      }
19.      p=pbuf_alloc(PBUF_RAW,len,PBUF_POOL); //pbufs 内存池分配 pbuf
20.      if(p!=NULL)
21.      {
22.          for(q=p;q!=NULL;q=q->next)
23.          {
24.              memcpy((u8_t*)q->payload,(u8_t*)&buffer[l], q->len);
25.              l=l+q->len;
26.          }
27.      }
28.      myfree(SRAMIN,buffer);                  //释放内存
29.      return p;
30.  }
```

在 ethernetif.c 中还剩下两个函数：ethernetif_input() 和 ethernetif_init()。ethernetif_input() 函数主要是对 low_level_input() 函数做封装，然后将接收到的数据送入指定的网卡结构中。ethernetif_input() 函数代码如下：

```
1.  //网卡接收数据(lwip 直接调用)
2.  //netif:网卡结构体指针
3.  //返回值:ERR_OK,发送正常
4.  //      ERR_MEM,发送失败
```

```

5.  err_t ethernetif_input(struct netif *netif)
6.  {
7.      err_t err;
8.      struct pbuf *p;
9.      p=low_level_input(netif); //调用 low_level_input 函数接收数据
10.     if(p==NULL) return ERR_MEM;
11.     err=netif->input(p, netif); //调用 netif 结构体中的 input 字段(一个函数)来处理数据包
12.     if(err!=ERR_OK)
13.     {
14.         LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input error\n"));
15.         pbuf_free(p);
16.         p = NULL;
17.     }
18.     return err;
19. }

```

ethernetif_init() 函数为 low_level_init() 函数的简单封装，并且初始化了 netif 的相关字段，ethernetif_init() 函数代码如下。

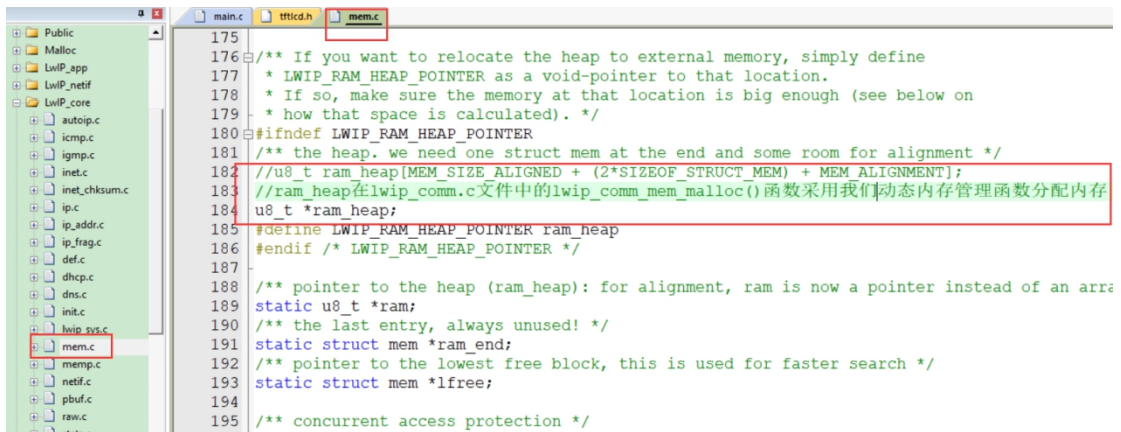
```

1.  //使用 low_level_init() 函数来初始化网络
2.  //netif:网卡结构体指针
3.  //返回值:ERR_OK, 正常
4.  //      其他, 失败
5.  err_t ethernetif_init(struct netif *netif)
6.  {
7.      LWIP_ASSERT("netif!=NULL", (netif!=NULL));
8.      #if LWIP_NETIF_HOSTNAME //LWIP_NETIF_HOSTNAME
9.          netif->hostname="lwip"; //初始化名称
10.     #endif
11.     netif->name[0]=IFNAME0; //初始化变量 netif 的 name 字段
12.     netif->name[1]=IFNAME1; //在文件外定义这里不用关心具体值
13.     netif->output=etharp_output; //IP 层发送数据包函数
14.     netif->linkoutput=low_level_output; //ARP 模块发送数据包函数
15.     low_level_init(netif); //底层硬件初始化函数
16.     return ERR_OK;
17. }

```

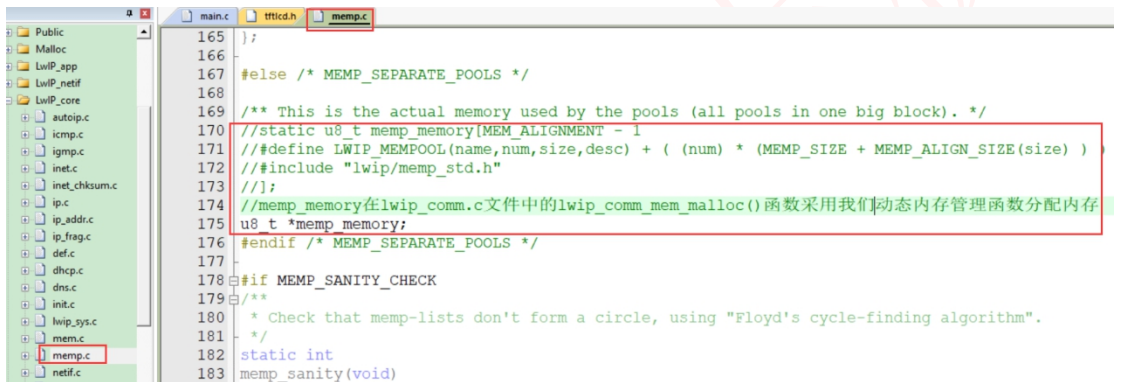
(3) 修改 mem.c 和 memp.c 文件

根据我们前面讲过的 LWIP 动态内存管理技术，我们知道 LWIP 有一个内存堆 ram_heap 和内存池 memp_memory，这两个是 LWIP 的内存来源。这两个分别在 mem.c 和 memp.c 中，我们将这两个数组改用我们自己编写的内存分配函数对其进行内存分配。在 mem.c 文件中我们将 ram_heap 数组注销掉，定义为指向 u8_t 的指针，如下图所示。



```
175
176 /** If you want to relocate the heap to external memory, simply define
177  * LWIP_RAM_HEAP_POINTER as a void-pointer to that location.
178  * If so, make sure the memory at that location is big enough (see below on
179  * how that space is calculated). */
180 #ifndef LWIP_RAM_HEAP_POINTER
181 /** the heap. We need one struct mem at the end and some room for alignment */
182 //u8_t ram_heap[MEM_SIZE ALIGNED + (2*SIZEOF_STRUCT_MEM) + MEM_ALIGNMENT];
183 //ram_heap在lwip_comm.c文件中的lwip_comm_mem_malloc()函数采用我们动态内存管理函数分配内存
184 u8_t *ram_heap;
185 #define LWIP_RAM_HEAP_POINTER ram_heap
186 #endif /* LWIP_RAM_HEAP_POINTER */
187
188 /** pointer to the heap (ram_heap): for alignment, ram is now a pointer instead of an array
189 static u8_t *ram;
190 /** the last entry, always unused! */
191 static struct mem *ram_end;
192 /** pointer to the lowest free block, this is used for faster search */
193 static struct mem *lfree;
194
195 /** concurrent access protection */
```

同样我们也将 memp.c 文件中将 memp_memory 数组屏蔽掉改为指针，如下图所示。



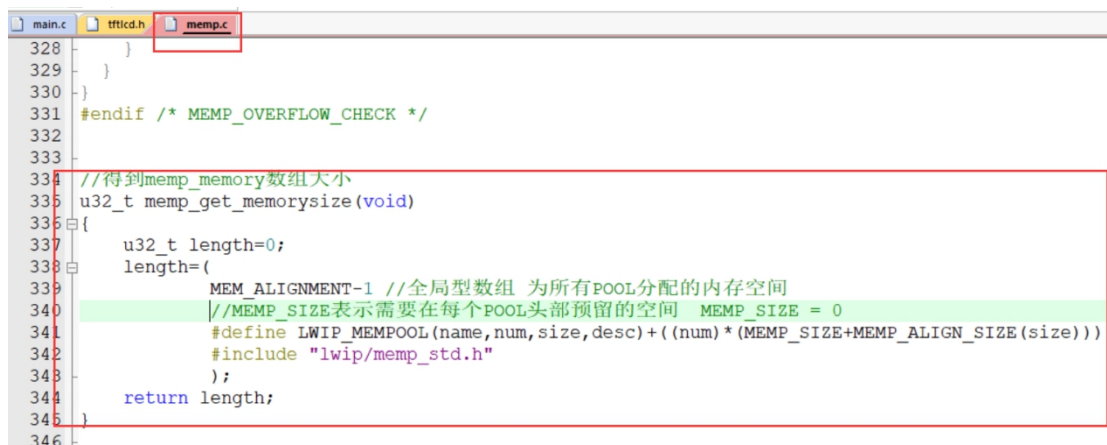
```
165
166
167 #else /* MEMP_SEPARATE_POOLS */
168
169 /** This is the actual memory used by the pools (all pools in one big block). */
170 //static u8_t memp_memory[MEM_ALIGNMENT - 1
171 //define LWIP_MEMPOOL(name,num,size,desc) + ( (num) * (MEM_SIZE + MEMP_ALIGN_SIZE(size)) )
172 //include "lwip/memp_std.h"
173 //];
174 //memp_memory在lwip_comm.c文件中的lwip_comm_mem_malloc()函数采用我们动态内存管理函数分配内存
175 u8_t *memp_memory;
176 #endif /* MEMP_SEPARATE_POOLS */
177
178 #if MEMP_SANITY_CHECK
179 /**
180  * Check that memp-lists don't form a circle, using "Floyd's cycle-finding algorithm".
181  */
182 static int
183 memp_sanity(void)
```

我们还要在 memp.c 文件中添加 memp_get_memorysize() 函数用来获取 memp_memory 数组的大小, memp_get_memorysize() 函数代码如下。

```
1. //得到 memp_memory 数组大小
2. u32_t memp_get_memorysize(void)
3. {
4.     u32_t length=0;
5.     length=(
6.         MEM_ALIGNMENT-1 //全局型数组 为所有 POOL 分配的内存空间
7.         //MEMP_SIZE 表示需要在每个 POOL 头部预留的空间 MEMP_SIZE = 0
8.         #define LWIP_MEMPOOL(name,num,size,desc)+((num)*(MEMP_SIZE+MEMP_ALIGN_SIZE(size)))
9.         #include "lwip/memp_std.h"
10.     );
11.     return length;
12. }
```

memp_memory 我们会在 lwip_comm.c 文件中使用动态内存管理函数为其分配内存。

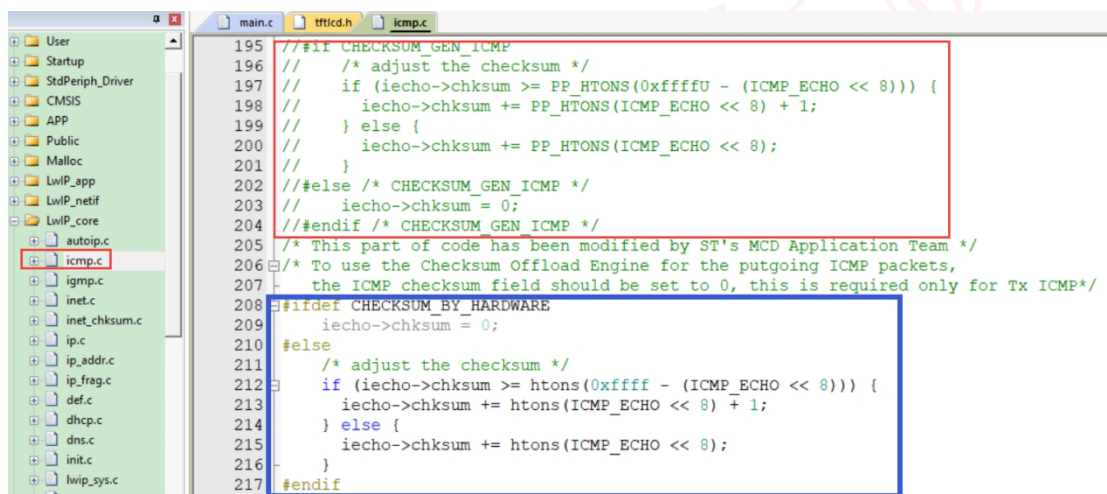
在 memp.c 文件中添加 memp_get_memorysize() 函数，如下图所示。



```
328 }
329 }
330 }
331 #endif /* MEMPOOL_OVERFLOW_CHECK */
332
333
334 //得到memp_memory数组大小
335 u32_t mem_get_memorysize(void)
336 {
337     u32_t length=0;
338     length=(
339         MEM_ALIGNMENT-1 //全局型数组 为所有POOL分配的内存空间
340         //MEM_SIZE表示需要在每个POOL头部预留的空间 MEM_SIZE = 0
341         #define LWIP_MEMPOOL(name,num,size,desc)+((num)*(MEM_SIZE+MEM_ALIGN_SIZE(size)))
342         #include "lwip/memp_std.h"
343     );
344     return length;
345 }
346
```

(4) 修改 icmp.c 文件

我们需要修改 icmp.c 文档使其支持硬件帧校验，修改部分如下图所示。



```
195 // #if CHECKSUM_GEN_ICMP
196 // /* adjust the checksum */
197 // if (iecho->chksum >= PP_HTONS(0xffff - (ICMP_ECHO << 8))) {
198 //     iecho->chksum += PP_HTONS(ICMP_ECHO << 8) + 1;
199 // } else {
200 //     iecho->chksum += PP_HTONS(ICMP_ECHO << 8);
201 // }
202 // #else /* CHECKSUM_GEN_ICMP */
203 //     iecho->chksum = 0;
204 // #endif /* CHECKSUM_GEN_ICMP */
205 /* This part of code has been modified by ST's MCD Application Team */
206 /* To use the Checksum Offload Engine for the outgoing ICMP packets,
207    the ICMP checksum field should be set to 0, this is required only for Tx ICMP*/
208 #ifdef CHECKSUM_BY_HARDWARE
209     iecho->chksum = 0;
210 #else
211     /* adjust the checksum */
212     if (iecho->chksum >= htons(0xffff - (ICMP_ECHO << 8))) {
213         iecho->chksum += htons(ICMP_ECHO << 8) + 1;
214     } else {
215         iecho->chksum += htons(ICMP_ECHO << 8);
216     }
217 #endif
```

图中红框部分为 icmp.c 的源码，我们将其注销掉，蓝框部分是我们需要添加进去的代码，这部分代码是由 ST 提供的。

3.2.7 LWIP 的裁剪与配置

在 LWIP 的源码中有个 opt.h 的文件，这个文件是裁剪和配置 LWIP 的，不过我们最好不要直接在 opt.h 里面做修改，我们可以打开 opt.h 文件看一下，里面的配置都是条件编译的，如果我们在其他地方有定义过的话那么在 opt.h 里面的定义不起作用了。所以我们可以新建一个.h 的文件来裁剪和配置 LWIP，我们前面提过在 LwIP->lwip_app->lwip_comm 下有一个 lwipopts.h 的文件，这个文件就是用来裁剪与配置 lwipopts.h 的，lwipopts.h 配置代码如下。

1. #ifndef __LWIPOPTS_H__

```

2.  #define __LWIPOPTS_H__
3.
4.  #define SYS_LIGHTWEIGHT_PROT    0
5.
6.  //NO_SYS==1:不使用操作系统
7.  #define NO_SYS                    1  //不使用 UCOS 操作系统
8.
9.  //使用 4 字节对齐模式
10. #define MEM_ALIGNMENT             4
11.
12.  //MEM_SIZE:heap 内存的大小,如果在应用中有大量数据发送的话这个值最好设置大一点
13. #define MEM_SIZE                  10*1024  //内存堆大小
14.
15.  //MEMP_NUM_PBUF:memp 结构的 pbuf 数量,如果应用从 ROM 或者静态存储区发送大量数据时,这个值应该设置大一点
16. #define MEMP_NUM_PBUF             10
17.
18.  //MEMP_NUM_UDP_PCB:UDP 协议控制块(PCB)数量.每个活动的 UDP"连接"需要一个 PCB.
19. #define MEMP_NUM_UDP_PCB         6
20.
21.  //MEMP_NUM_TCP_PCB:同时建立激活的 TCP 数量
22. #define MEMP_NUM_TCP_PCB         10
23.
24.  //MEMP_NUM_TCP_PCB_LISTEN:能够监听的 TCP 连接数量
25. #define MEMP_NUM_TCP_PCB_LISTEN  6
26.
27.  //MEMP_NUM_TCP_SEG:最多同时在队列中的 TCP 段数量
28. #define MEMP_NUM_TCP_SEG         20
29.
30.  //MEMP_NUM_SYS_TIMEOUT:能够同时激活的 timeout 个数
31. #define MEMP_NUM_SYS_TIMEOUT     5
32.
33.
34.  /* ----- Pbuf 选项----- */
35.  //PBUF_POOL_SIZE:pbuf 内存池个数.
36. #define PBUF_POOL_SIZE           10
37.
38.  //PBUF_POOL_BUFSIZE:每个 pbuf 内存池大小.
39. #define PBUF_POOL_BUFSIZE        1500
40.
41.
42.  /* ----- TCP 选项----- */
43. #define LWIP_TCP                  1  //为 1 是使用 TCP
44. #define TCP_TTL                   255//生存时间
45.

```

```

46.  /*当 TCP 的数据段超出队列时的控制位,当设备的内存过小的时候此项应为 0*/
47.  #define TCP_QUEUE_OOSEQ          0
48.
49.  //最大 TCP 分段
50.  #define TCP_MSS                    (1500 - 40)    //TCP_MSS = (MTU - IP 报头大小 - TCP 报头大小)
51.
52.  //TCP 发送缓冲区大小(bytes).
53.  #define TCP_SND_BUF                (4*TCP_MSS)
54.
55.  //TCP_SND_QUEUELEN: TCP 发送缓冲区大小(pbuf).这个值最小为(2 * TCP_SND_BUF/TCP_MSS)
56.  #define TCP_SND_QUEUELEN          (4* TCP_SND_BUF/TCP_MSS)
57.
58.  //TCP 发送窗口
59.  #define TCP_WND                    (2*TCP_MSS)
60.
61.
62.  /* ----- ICMP 选项----- */
63.  #define LWIP_ICMP                  1 //使用 ICMP 协议
64.
65.  /* ----- DHCP 选项----- */
66.  //当使用 DHCP 时此位应该为 1,LwIP 0.5.1 版本中没有 DHCP 服务.
67.  #define LWIP_DHCP                  1
68.
69.  /* ----- UDP 选项 ----- */
70.  #define LWIP_UDP                    1 //使用 UDP 服务
71.  #define UDP_TTL                    255 //UDP 数据包生存时间
72.
73.
74.  /* ----- Statistics options ----- */
75.  #define LWIP_STATS                  0
76.  #define LWIP_PROVIDE_ERRNO 1
77.
78.
79.  /*
80.   -----
81.   ----- SequentialAPI 选项-----
82.   -----
83.  */
84.
85.  //LWIP_NETCONN==1:使能 NETCON 函数(要求使用 api_lib.c)
86.  #define LWIP_NETCONN                0
87.
88.  /*
89.   -----

```

```

90.  ----- Socket API 选项-----
91.  -----
92.  */
93.  //LWIP_SOCKET==1:使能 Socket API(要求使用 sockets.c)
94.  #define LWIP_SOCKET                0
95.
96.  #define LWIP_COMPAT_MUTEX          1
97.
98.  #define LWIP_SO_RCVTIMEO           1 //通过定义 LWIP_SO_RCVTIMEO 使能 netconn 结构体中 recv_timeout,使用
    recv_timeout 可以避免阻塞线程
99.
100.
101.  /*
102.  -----
103.  ----- Lwip 调试选项-----
104.  -----
105.  */
106.  // #define LWIP_DEBUG                1 //开启 DEBUG 选项
107.
108.  #define ICMP_DEBUG                  LWIP_DBG_OFF //开启/关闭 ICMPdebug
109.
110.  #endif /* __LWIPOPTS_H__ */

```

我们可以看到 lwipopts.h 中有很多的宏定义，每个宏定义后面已经给出了具体的解释，大家可以参考一下，这里我们只是给出了一个参考配置，大家在使用过程中一定要按照自己的需求来编写 lwipopts.h 里面的内容。

3.3 硬件设计

前面我们已经介绍了 PZ-ENC28J60 模块的硬件电路。要将 PZ-ENC28J60 以太网模块连接到 STM32 开发板中，可使用杜邦线按照下列方式连接：（PZ-ENC28J60 以太网模块-->STM32 IO 口）

```

3. 3V-->3. 3V
GND-->GND
RST-->PG8
INT-->PG6
CS-->PG7
SCK-->PB13

```

MOSI-->PB15

MISO-->PB14

3.4 软件设计

经过上面几节的讲解，LWIP 移植部分已经完成，在本节我们可以编写 main.c 文件来测试移植是否成功，在 main.c 文件中我们有两个函数 show_address() 和 main() 函数，show_address() 函数用来在 LCD 上显示一些提示信息，比如 MAC 地址、IP 地址、子网掩码、默认网关等信息。我们重点讲解一下 main 函数，main 函数代码如下。

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "malloc.h"
8.  #include "enc28j60.h"
9.  #include "lwip/netif.h"
10. #include "lwip_comm.h"
11. #include "lwipopts.h"
12. #include "time.h"
13.
14.
15. //在 LCD 上显示地址信息
16. //mode:1 显示 DHCP 获取到的地址
17. //    其他 显示静态地址
18. void show_address(u8 mode)
19. {
20.     u8 buf[30];
21.     if(mode==1)
22.     {
23.         sprintf((char*)buf,"MAC      :%d.%d.%d.%d.%d.%d",lwipdev.mac[0],lwipdev.mac[1],lwipdev.mac[2],lwipdev.mac[3],lwipdev.mac[4],lwipdev.mac[5]); //打印 MAC 地址
24.         LCD_ShowString(30,130,210,16,16,buf);
25.         sprintf((char*)buf,"DHCP IP:%d.%d.%d.%d",lwipdev.ip[0],lwipdev.ip[1],lwipdev.ip[2],lwipdev.ip[3]);
26.         //打印动态 IP 地址
27.         LCD_ShowString(30,150,210,16,16,buf);
28.         sprintf((char*)buf,"DHCP GW:%d.%d.%d.%d",lwipdev.gateway[0],lwipdev.gateway[1],lwipdev.gateway[2],lwipdev.gateway[3]); //打印网关地址
```

```

28.         LCD_ShowString(30,170,210,16,16,buf);
29.         sprintf((char*)buf, "DHCP IP:%d.%d.%d.%d", lwipdev.netmask[0],lwipdev.netmask[1],lwipdev.netmask[2],
        lwipdev.netmask[3]); //打印子网掩码地址
30.         LCD_ShowString(30,190,210,16,16,buf);
31.     }
32.     else
33.     {
34.         sprintf((char*)buf, "MAC      :%d.%d.%d.%d.%d.%d", lwipdev.mac[0],lwipdev.mac[1],lwipdev.mac[2],lwip
        dev.mac[3],lwipdev.mac[4],lwipdev.mac[5]); //打印 MAC 地址
35.         LCD_ShowString(30,130,210,16,16,buf);
36.         sprintf((char*)buf, "Static IP:%d.%d.%d.%d", lwipdev.ip[0],lwipdev.ip[1],lwipdev.ip[2],lwipdev.ip[3])
        ;
        //打印动态 IP 地址
37.         LCD_ShowString(30,150,210,16,16,buf);
38.         sprintf((char*)buf, "Static GW:%d.%d.%d.%d", lwipdev.gateway[0],lwipdev.gateway[1],lwipdev.gateway[2]
        ,lwipdev.gateway[3]); //打印网关地址
39.         LCD_ShowString(30,170,210,16,16,buf);
40.         sprintf((char*)buf, "Static IP:%d.%d.%d.%d", lwipdev.netmask[0],lwipdev.netmask[1],lwipdev.netmask[2]
        ,lwipdev.netmask[3]); //打印子网掩码地址
41.         LCD_ShowString(30,190,210,16,16,buf);
42.     }
43. }
44.
45. int main()
46. {
47.     u32 i=0;
48.
49.     SysTick_Init(72);
50.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分 2 组
51.     LED_Init();
52.     USART1_Init(115200);
53.     TFTLCD_Init(); //LCD 初始化
54.     KEY_Init();
55.     TIM3_Init(1000,719); //定时器 3 频率为 100hz
56.     my_mem_init(SRAMIN); //初始化内部内存池
57.
58.     FRONT_COLOR=RED; //设置字体为红色
59.     LCD_ShowString(30,30,200,16,16,"ENC28J60 LwIP Test");
60.     LCD_ShowString(30,50,200,16,16,"www.prechin.cn");
61.     while(lwip_comm_init()) //lwip 初始化
62.     {
63.         LCD_ShowString(30,110,200,20,16,"LWIP Init Falied!");
64.         delay_ms(1200);
65.         LCD_Fill(30,110,230,130,WHITE); //清除显示
66.         LCD_ShowString(30,110,200,16,16,"Retrying...");

```

```

67.     }
68.     LCD_ShowString(30,110,200,20,16,"LWIP Init Success!");
69.     LCD_ShowString(30,130,200,16,16,"DHCP IP configing...");
70.     #if LWIP_DHCP    //使用 DHCP
71.         while((lwipdev.dhcpstatus!=2)&&(lwipdev.dhcpstatus!=0XFF))//等待 DHCP 获取成功/超时溢出
72.         {
73.             lwip_periodic_handle(); //LWIP 内核需要定时处理的函数
74.         }
75.     #endif
76.     show_address(lwipdev.dhcpstatus);    //显示地址信息
77.     while(1)
78.     {
79.         lwip_periodic_handle(); //LWIP 内核需要定时处理的函数
80.         i++;
81.         if(i==50000)
82.         {
83.             LED1=!LED1;
84.             i=0;
85.         }
86.     }
87. }

```

在 main 函数中首先完成对外设的初始化，然后调用 lwip_comm_init 函数初始化 LWIP，如果使用 DHCP 的话就先等待 DHCP 获取 IP 地址完成，然后在 LCD 上显示地址信息，如果 DHCP 获取地址失败的话将使用我们的默认的地址，最后我们在一个 while 循环中循环调用 lwip_periodic_handle() 和 lwip_pkt_handle() 这两个函数。在 main 函数中我们要注意到不管是在等待 DHCP 完成还是 DHCP 成功后我们都要周期性调用 lwip_periodic_handle() 和 lwip_pkt_handle() 这两个函数，因为在这个函数中周期性调用协议栈内核的一些定时函数以满足 LWIP 的内核要求。

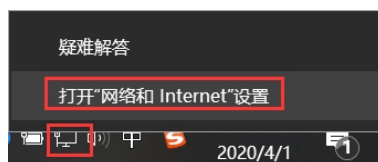
3.5 实验现象

(1) 连接设置

在代码编译成功以后，我们下载代码到开发板中，通过网线连接开发板到路由器上，如果没有路由器的话也可以直接连接到电脑的网口上。但是如果连接到电脑的网口上那么开发板就不能使用 DHCP 功能，需要使用静态地址，我们例程中的默认静态 IP 地址：192.168.1.30，子网掩码：255.255.255.0，默认网关：

192.168.1.1。下面我们就以直接连接电脑网口为例，连接上电脑端的网口以后我们还需要更改一下电脑的网络设置，步骤如下，此处以 WIN10 系统为例。

①打开“网络和 Internet”设置，可以在电脑右下角网络连接图标上鼠标右键，选择打开“网络和 Internet”设置，如下图所示。



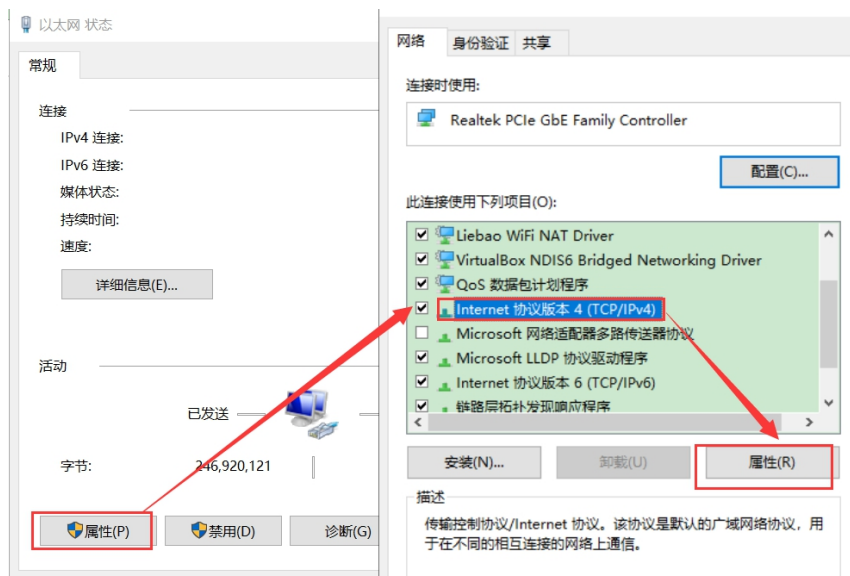
②选择“以太网”中的“网络和共享中心”，如下图所示。



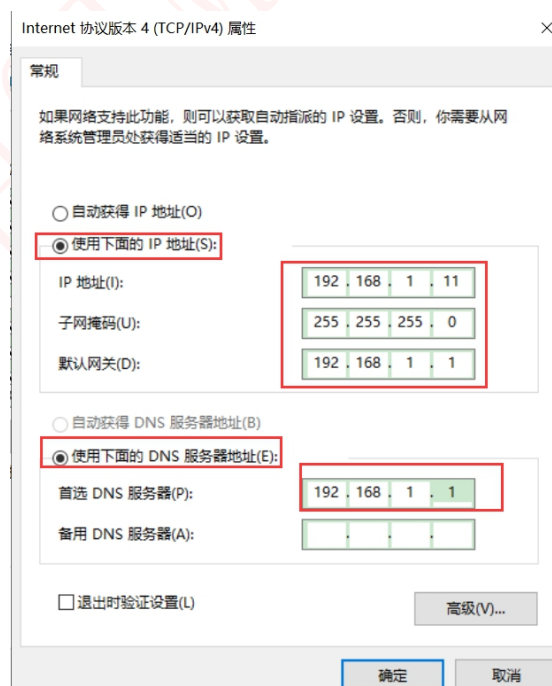
③在“网络和共享中心”窗口中，打开我们电脑的本地以太网连接，如下所示：



④然后在弹出的对话框中，点击“属性”，选择“TCP/IPv4”，点击属性，如下所示：



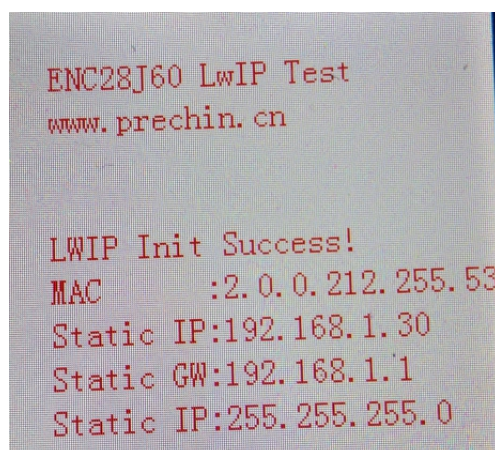
⑤点击“属性”后出现如下图所示对话框，选择“使用下面的 IP 地址”和“使用下面的 DNS 服务器地址”。在 IP 地址栏填入：192.168.1.x(x 为 2-254)，子网掩码：255.255.255.0 默认网关：192.168.1.1，首选 DNS 服务器：192.168.1.1，最后记得点击“确定”。注意！电脑的 IP 地址一定要和开发板的 IP 地址在一个网络内！以后我们的实验都是连接到路由器上的，如果没有路由器的话都可以使用上面这种方法这种方法完成实验，只是不能使用 DHCP 服务。



(2) 验证测试

打开串口调试助手，复位一下开发板。这里我们开启了 DHCP，大家也可以

自行尝试一下关闭 DHCP 使用静态 IP 地址，下载完成后 LCD 显示如下图所示。



可以看到此时直接与电脑网口相连,只能使用静态 IP 地址为:192.168.1.30, 默认网关为:192.168.1.1, 子网掩码为:255.255.255.0。在电脑上 ping 开发板的 IP 地址,结果如下图所示。(能 Ping 通表示硬件连接是 OK 的)

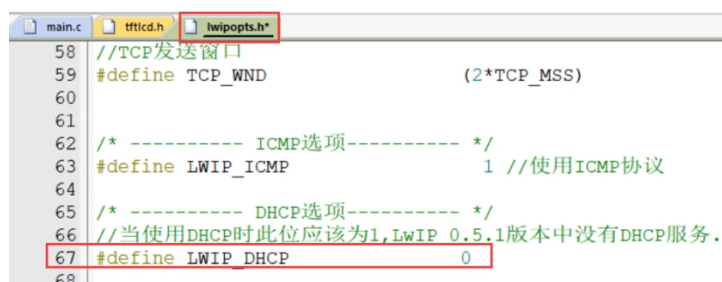
```
命令提示符
C:\Users\YZ>ping 192.168.1.30

正在 Ping 192.168.1.30 具有 32 字节的数据:
来自 192.168.1.30 的回复: 字节=32 时间=1ms TTL=255
来自 192.168.1.30 的回复: 字节=32 时间=1ms TTL=255
来自 192.168.1.30 的回复: 字节=32 时间=1ms TTL=255
来自 192.168.1.30 的回复: 字节=32 时间=1ms TTL=255

192.168.1.30 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 1ms, 最长 = 1ms, 平均 = 1ms

C:\Users\YZ>
```

注意: 如果开发板是和电脑的网口相连,而且开启了 DHCP,开发板就会等待 DHCP 完成,这个时候由于电脑没有 DHCP 服务因此会等待很久,直到 DHCP 超时使用默认地址。如果不愿意等,可以在 lwipopts.h 文件中设置 LWIP_DHCP 为 0。



4 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）